Statistical Methods for single cell data analysis 3

Yongjin Park, UBC Path + Stat, BC Cancer

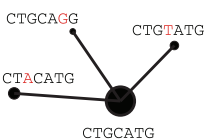
17 March 2025

Source code available:

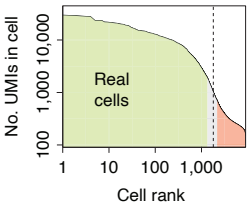
`https://github.com/stat540-UBC/lectures`

Overview of single-cell data analysis

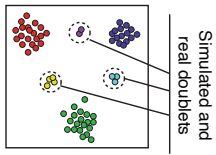
Alignment and molecular counting



Cell filtering and quality control



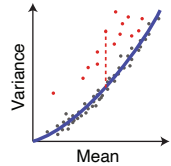
Doublet scoring



Cell size estimation

Cells	c_1	c_2	c_3
$Gene_1$	2	4	20
$Gene_2$	1	2	10
$Gene_3$	3	6	30
Cell depth:	6	12	60

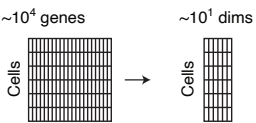
Gene variance analysis



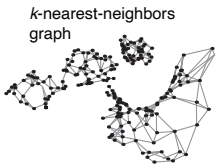
Karchenko, *Nature Methods* (2021)

Overview of single-cell data analysis cont'd

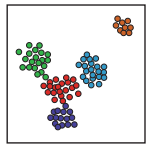
Reduction to a medium-dimensional space



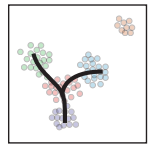
Manifold representation



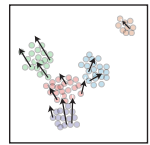
Clustering and differential expression



Trajectories



Velocity estimation



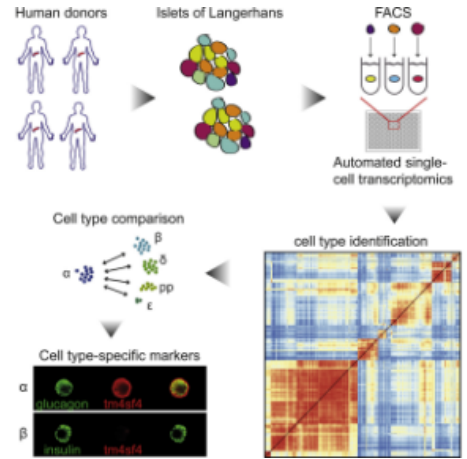
Karchenko, *Nature Methods* (2021)

The goal of modelling in high-dimensional space

The goal is to model two types of relationships:

- Relationship between dimensions/features (genes)
- Relationship between samples/data points (cells)
- One of the most fundamental relationships is co-variation.

Example: scRNA-seq data of human pancreatic cells



Muraro *et al. Cell Systems* (2016)

- 19,140 genes/features/rows
- 3,072 cells/columns
- 12,442,034 non-zero elements
- $\approx$ 21 % non-zero elements

What is a latent variable/factor model?

Examples:

- Principal Component Analysis
- Non-negative matrix factorization
- (…)

What is Unsupervised Learning?

Supervised learning

- Input: data $\mathcal{X} = \{\mathbf{x}_i, ...\}$
- Input 2: label $\mathcal{Y} = \{y_i, ...\}$
- Goal: learn $f : \mathcal{X} \to \mathcal{Y}$

X: gene expression samples; Y: disease labels

Unsupervised learning

- Input: data $\mathcal{X} = \{\mathbf{x}_i, ...\}$
- Goal 1: learn $f : \mathcal{Z} \to \mathcal{X}$
- Goal 2: hidden/latent states, $\mathcal{Z} = \{\mathbf{z}_i, ...\}$

X: gene expression matrices

Today's lecture

1. Representation learning
2. Warm-up: Clustering is an unsupervised learning
3. VAE: supervise approaches to unsupervised learning
4. Topic modelling in VAE framework

How can we obtain a good representation of data?

Two strategies to learn a “good” representation

Feature engineering

Construct useful combinations of variables, feature engineering

- Principal component analysis (matrix factorization)
- Pattern recognition
- Clustering (average patterns $\approx$ combined features/data)

Manifold learning

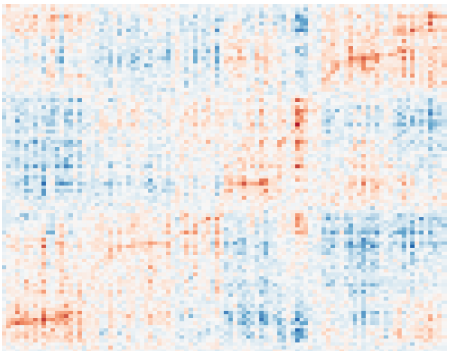
Change the manifold (a coordinate system) of data

- Probabilistic topic models: word frequency $\longrightarrow$ simplex
- Angular space $\longrightarrow$ Euclidean space
- Observed high-dim. $\longrightarrow$ interpretable ones

We use both strategies in practice.

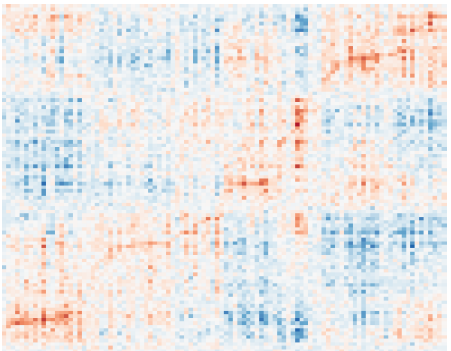
Feature engineering by combining multiple vars.

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$



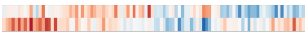
Feature engineering by combining multiple vars.

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$



FEATURES

X

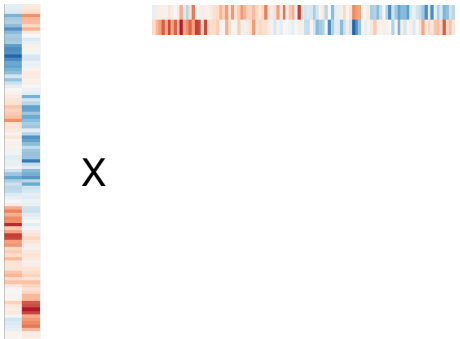


feature loading

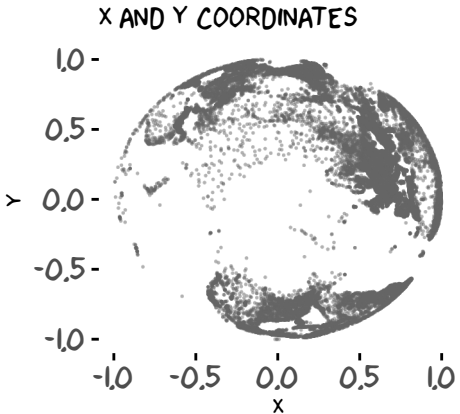
Feature engineering

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$

- We call each column of U and V a factor (row/column)
- One factor corresponds to the activities/patterns/expressions of 100 genes/samples
- We will discuss further later.

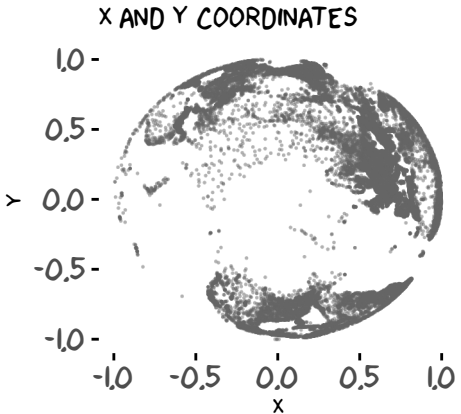


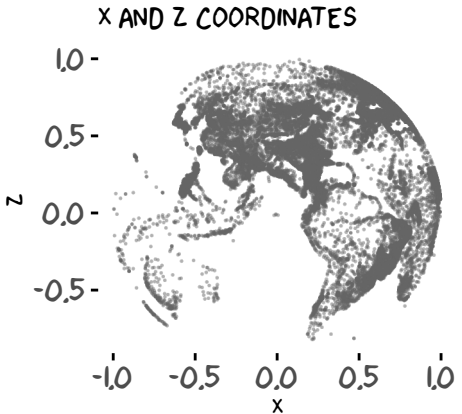
Manifold learning changes the coordinate system of data



Data from https://simplemaps.com/data/world-cities

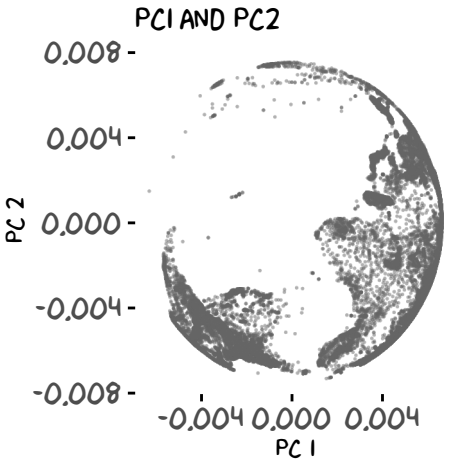
Manifold learning changes the coordinate system of data





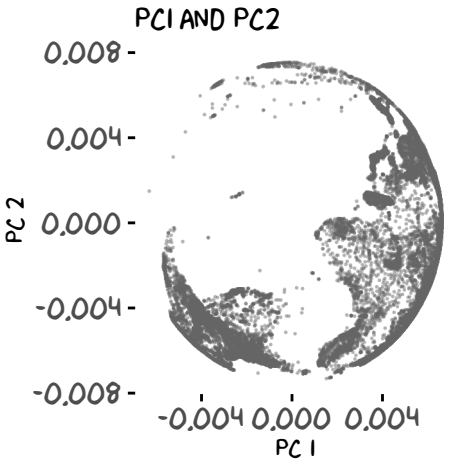
Data from https://simplemaps.com/data/world-cities

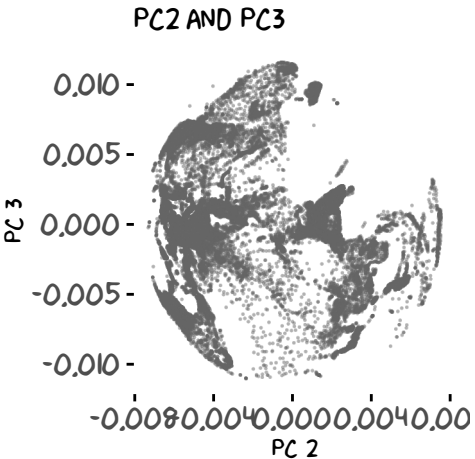
Manifold learning changes the coordinate system of data



Data from https://simplemaps.com/data/world-cities

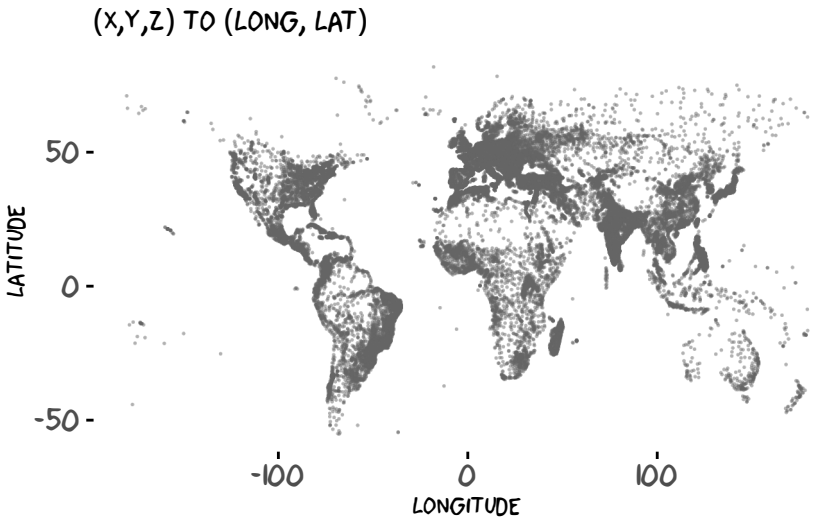
Manifold learning changes the coordinate system of data





Data from https://simplemaps.com/data/world-cities

Manifold learning changes the coordinate system of data



Representation learning

Feature engineering

- Combine samples based on similarity to learn common features
- Combine features/genes/proteins to create a new factor
- Matrix factorization to capture factors that explain large variation

Manifold learning

- Change the coordinate system
- What's not separable in one space can be spread out in another one
- Apply non-linear functions to a set of features

Today's lecture

1. Representation learning
2. Warm-up: Clustering is an unsupervised learning
3. VAE: supervise approaches to unsupervised learning
4. Topic modelling in VAE framework

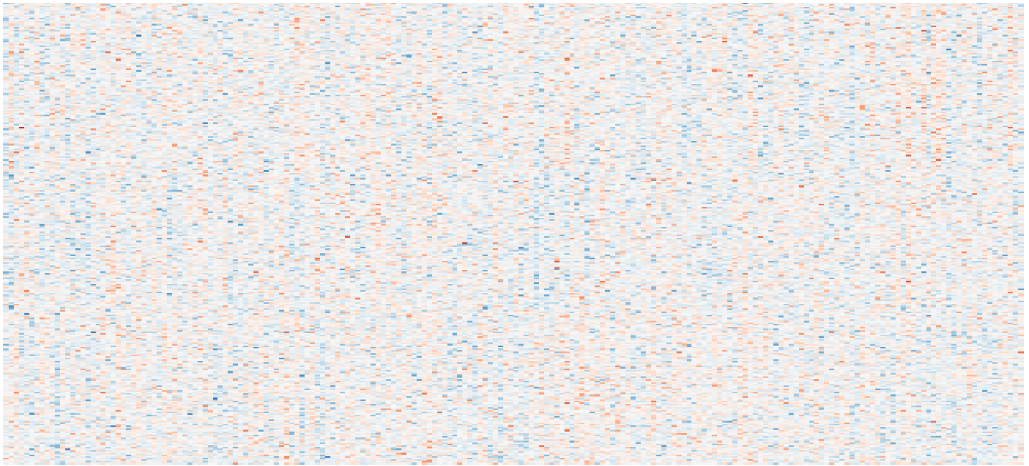
Warm-up: Clustering is an unsupervised learning

The goal of this clustering section

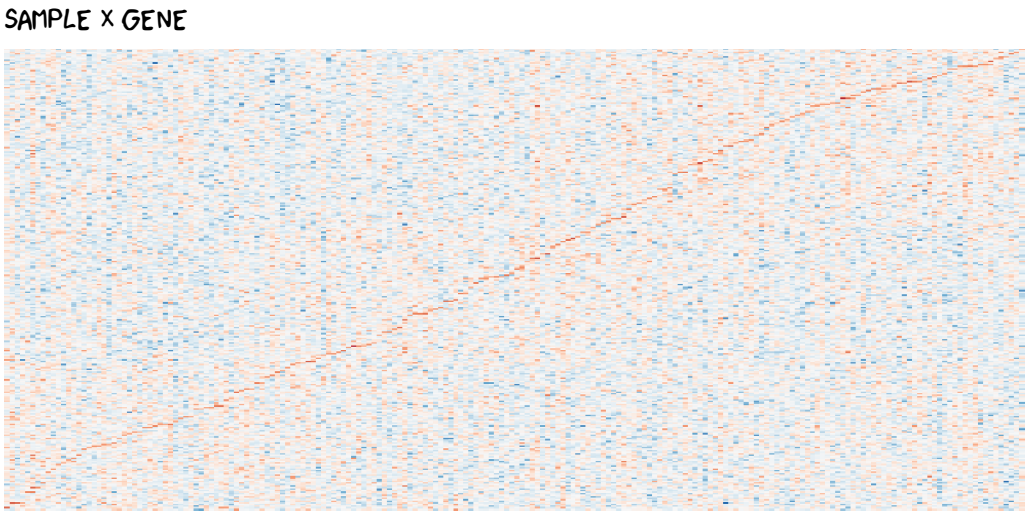
- Model-based unsupervised learning
  - Latent variable model
- Expectation Maximization
- Hands-on experience with clustering algorithm

Clustering: Find commonality across rows and cols

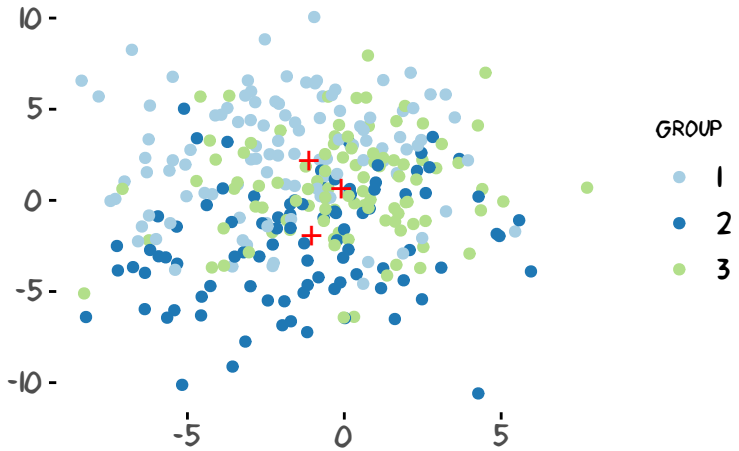
SAMPLE X GENE



Clustering: Find commonality across rows and cols



What do we want to know from data?

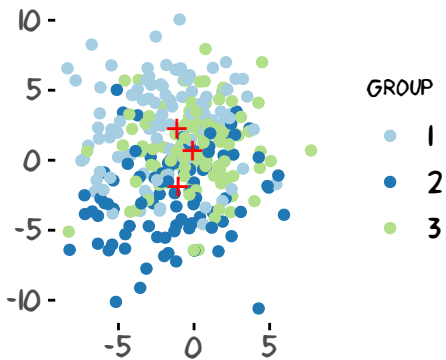


Two goals: recover (1) group membership and (2) the centroids (red marks)

A chicken-and-egg problem: guessing latent membership vs. parametric inference

- If we knew the membership of all the points, we can simply estimate the centre (e.g., taking sample mean within each cluster)
- If we knew the centre coordinates, we would be able to assign points to most probable groups easily based on distance from the centre points.
- Statistical answer: Solve the underlying inference problem.
- To a parameter estimator, the membership assignments are *hidden* (latent).

Let's think about the data-generating process to "reverse" it

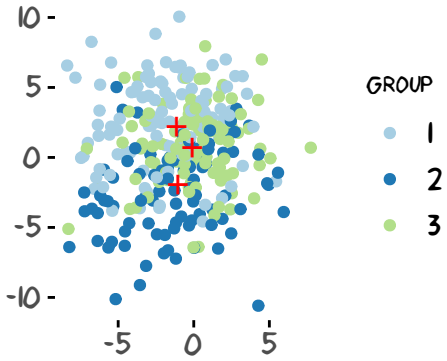


Latent Variable Model

What could have been done to generate observe data?

- Latent membership: Z_{ik}
- Model parameters: μ_k and σ_k

Gaussian Mixture Model (k-means)



GMM data generating process

1. Initialize μ_k (the centre of each group) and σ_k (the spread within each group)
2. Randomly assign group membership, $Z_{ik} = 1$ iff a point i belongs to a group k.
3. Generate: $\mathbf{x}_i | Z_{ik} = 1, \mu_k \sim \mathcal{N}\left(\mu_k, \sigma^2 I\right)$

How do we infer Z and μ, σ?

Expectation Maximization for GMM MLE

$$J \equiv \log \prod_{i=1}^{n} p(\mathbf{x}_i | \mu, \sigma)$$

Expectation Maximization for GMM MLE

$$
\begin{aligned}
J &\equiv \log \prod_{i=1}^{n} p(\mathbf{x}_i|\mu, \sigma) \\
&= \log \prod_{i=1}^{n} \sum_{Z} p(\mathbf{x}_i|Z, \mu, \sigma) p(Z)
\end{aligned}
$$

It might be difficult to enumerate all the Z's...

Expectation Maximization for GMM MLE

$$J \equiv \log \prod_{i=1}^{n} p(\mathbf{x}_i|\mu, \sigma)$$

It might be difficult to enumerate all the Z's...

so let's introduce some other distributions that will help our "guessing" work, which we call it $q(Z)$

EM algorithm: What is the best way to guess latent variables?

$$\sum_i \log p(\mathbf{x}_i|\mu,\sigma) \quad = \quad \sum_{i=1}^{n} \log \sum_{Z_i} \frac{q(Z_i)}{q(Z_i)} p(\mathbf{x}_i|Z_i,\mu,\sigma)p(Z_i)$$

EM algorithm: What is the best way to guess latent variables?

$$\sum_i \log p(\mathbf{x}_i|\mu,\sigma) = \sum_{i=1}^{n} \log \sum_{Z_i} \frac{q(Z_i)}{q(Z_i)} p(\mathbf{x}_i|Z_i,\mu,\sigma)p(Z_i)$$

$$\geq_{\text{Jensen}} \sum_{i=1}^{n} \sum_{Z_i} q(Z_i) \log \frac{p(\mathbf{x}_i|Z_i,\mu,\sigma)p(Z_i)}{q(Z_i)}$$

EM algorithm: What is the best way to guess latent variables?

$$\sum_i \log p(\mathbf{x}_i|\mu,\sigma) = \sum_{i=1}^{n} \log \sum_{Z_i} \frac{q(Z_i)}{q(Z_i)} p(\mathbf{x}_i|Z_i,\mu,\sigma)p(Z_i)$$

$$\underset{\text{Jensen}}{\geq} \sum_{i=1}^{n} \sum_{Z_i} q(Z_i) \log \frac{p(\mathbf{x}_i|Z_i,\mu,\sigma)p(Z_i)}{q(Z_i)}$$

What is the best $q(Z)$?

EM: optimal E-step is to take the posterior probability

If $q(Z) = p(Z|\mathbf{x}_i, \mu, \sigma)$ (by Bayes rule),

$$= \sum_{i}^{n} \sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma) \log \frac{p(\mathbf{x}_i|Z_i, \mu, \sigma)p(Z_i)}{p(Z_i|\mathbf{x}_i\mu, \sigma)}$$

EM: optimal E-step is to take the posterior probability

If $q(Z) = p(Z|\mathbf{x}_i, \mu, \sigma)$ (by Bayes rule),

$$= \sum_i^n \sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma) \log \frac{p(\mathbf{x}_i|Z_i, \mu, \sigma)p(Z_i)}{p(Z_i|\mathbf{x}_i\mu, \sigma)}$$

$$= \sum_i^n \sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma) \log \frac{p(\mathbf{x}_i, Z_i|\mu, \sigma)p(\mathbf{x}_i|\mu, \sigma)}{p(\mathbf{x}_i, Z_i|\mu, \sigma)}$$

EM: optimal E-step is to take the posterior probability

If $q(Z) = p(Z|\mathbf{x}_i, \mu, \sigma)$ (by Bayes rule),

$$
\begin{aligned}
&= \sum_i^n \sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma) \log \frac{p(\mathbf{x}_i|Z_i, \mu, \sigma)p(Z_i)}{p(Z_i|\mathbf{x}_i\mu, \sigma)} \\
&= \sum_i^n \sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma) \log \frac{p(\mathbf{x}_i, Z_i|\mu, \sigma)p(\mathbf{x}_i|\mu, \sigma)}{p(\mathbf{x}_i, Z_i|\mu, \sigma)} \\
&= \sum_{i=1}^n \underbrace{\left[\sum_{Z_i} p(Z_i|\mathbf{x}_i, \mu, \sigma)\right]}_{=1} \log p(\mathbf{x}_i|\mu, \sigma)
\end{aligned}
$$

Expectation Maximization algorithm = expected MLE

$$\log p(X|\mu,\sigma) = \log \sum_Z p(X,Z|\mu,\sigma) \tag{1}$$

$$\geq_{\text{Jensen}} \sum_Z \underbrace{p(Z|X,\mu,\sigma)}_{\text{E-step}} \underbrace{\log p(X|Z,\mu,\sigma)}_{\text{M-step}} \tag{2}$$

$$= \underbrace{\mathbb{E}_{p(Z|X,\mu,\sigma)}}_{\text{E-step}} \left[\underbrace{\log p(X|Z,\mu,\sigma)}_{\text{M-step}} \right] \tag{3}$$

(Z is discrete, e.g., a membership indicator)

Solution: **Maximize** the lower bound by taking **the expectation** over the posterior probability.

EM algorithm of GMM: E-step

Log-likelihood under some group (μ_k and σ_k):

$\log p(\mathbf{x}_i|\mu_k, \sigma_k) = \log \mathcal{N}(\mathbf{x}_i|\mu_k, \sigma_k)$

How to estimate the posterior?

$$p(Z_{ik}|\mathbf{x}_i, \mu, \sigma) = \frac{\exp\{\log p(\mathbf{x}_i|\mu_k, \sigma_k)\}}{\sum_{k'} \exp\{\log p(\mathbf{x}_i|\mu_{k'}, \sigma_{k'})\}}$$

Remark: We can stochastically sample $Z_{ik} = 1$ with the posterior probability.

EM algorithm of GMM: M-step

Maximization step to optimize model parameters

Let this expected lower-bound (ELBO)

$$\mathcal{L}(\mathbf{x}_i; \{\mu_k\}, \{\sigma_k\}) = \sum_{i=1}^{n} \sum_{k=1}^{K} Z_{ik} \log p(\mathbf{x}_i | Z_{ik}, \mu_k, \sigma_k)$$

Given Z, what are the unknown? We can take gradient steps (e.g., `torch`)

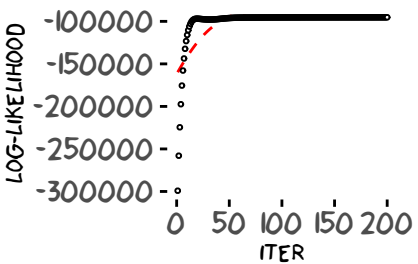
$$\mu_k^{(t)} \leftarrow \mu_k^{(t-1)} + \rho \nabla_{\mu_k} \sum_i \mathcal{L}(\mathbf{x}_i)$$

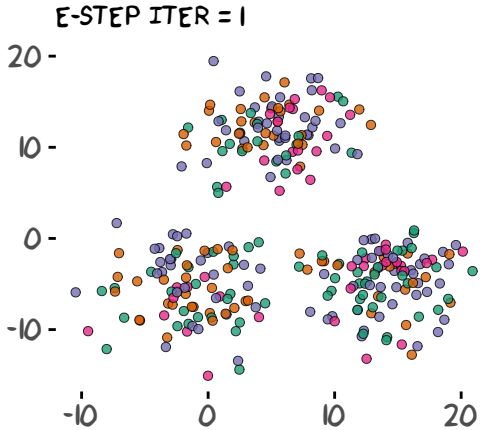
$$\sigma_k^{(t)} \leftarrow \sigma_k^{(t-1)} + \rho \nabla_{\sigma_k} \sum \mathcal{L}(\mathbf{x}_i)$$

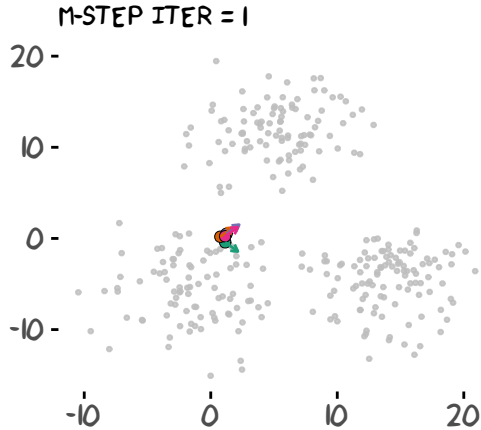
Alternate E- and M-step until convergence

```
for(tt in 1:100){
    latent.assign <- take.estep()
    z <- nnf_one_hot(latent.assign)
    llik <- take.mstep(z)
}
```

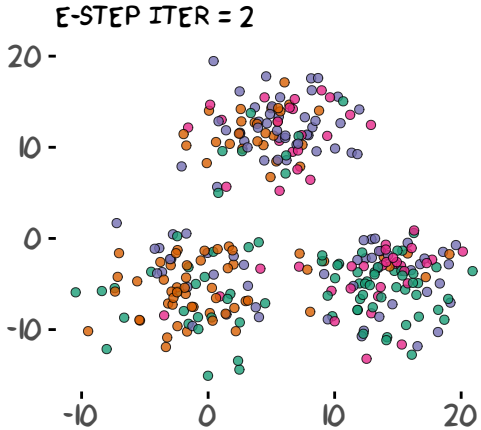
Find the details here:
https://github.com/STAT540-UBC/lectures

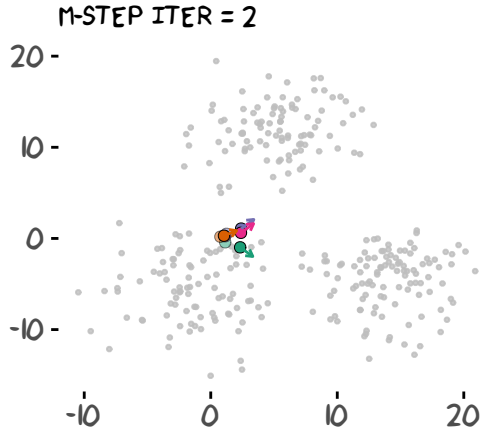




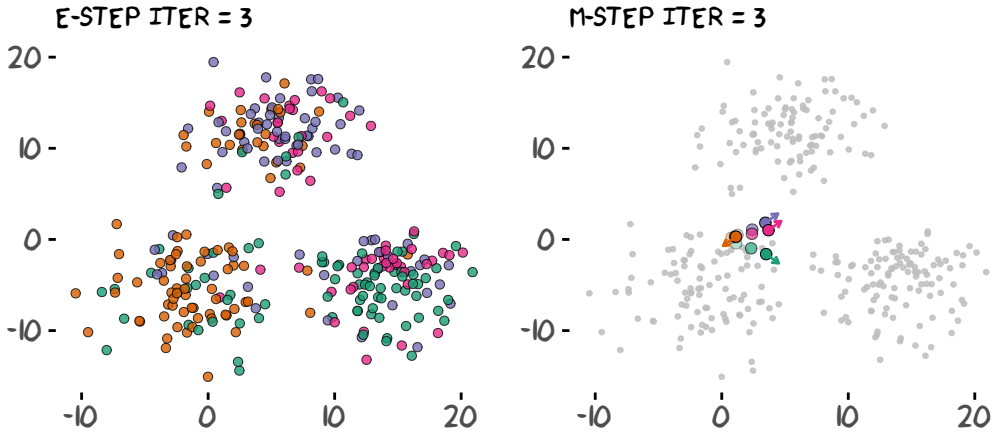


- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

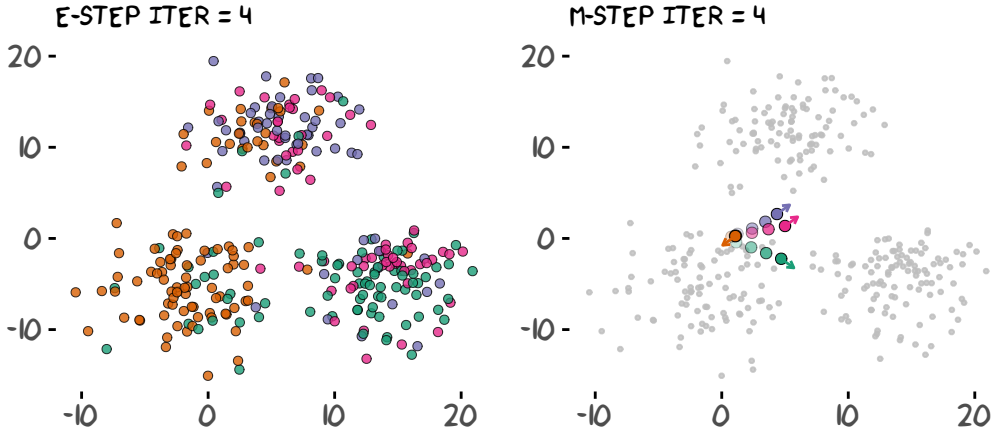




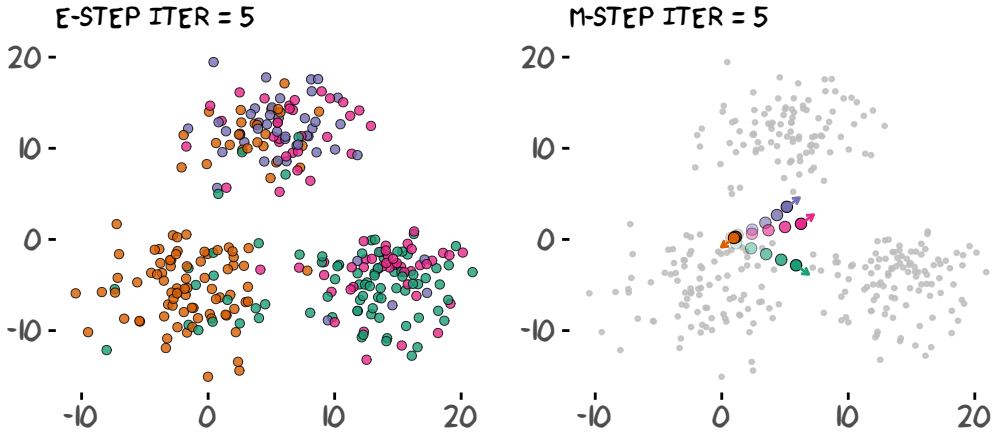
- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



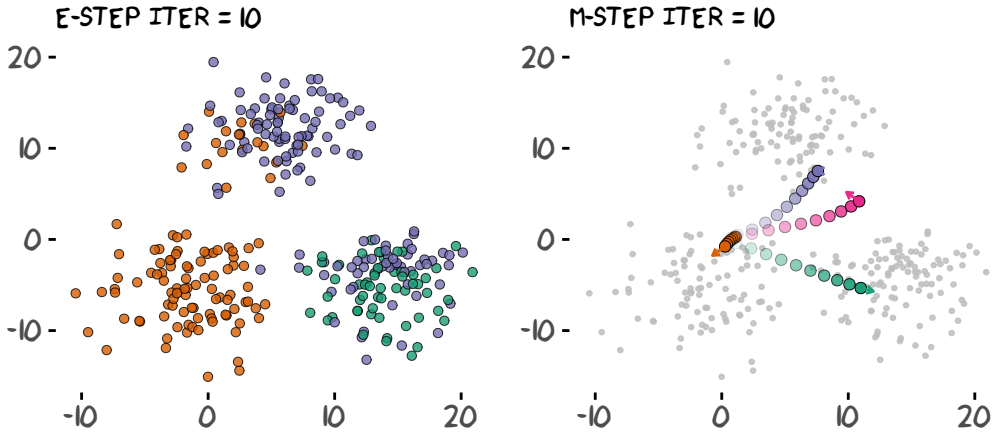
- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



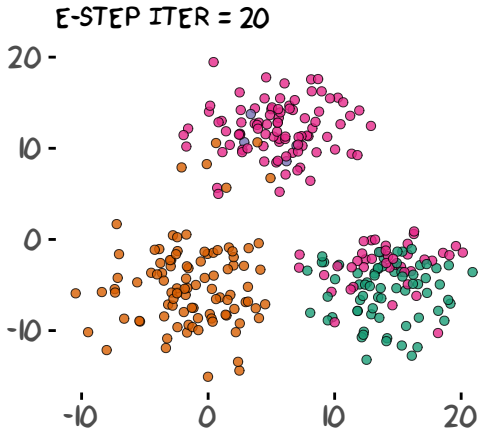
- Arrows: stochastic gradient $\nabla\mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

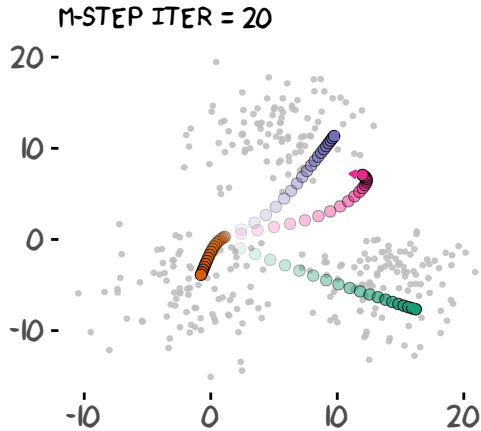


- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

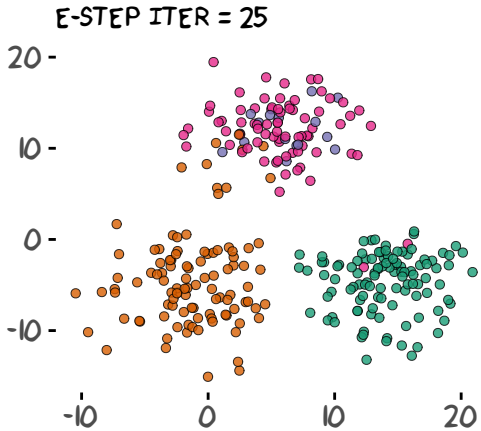


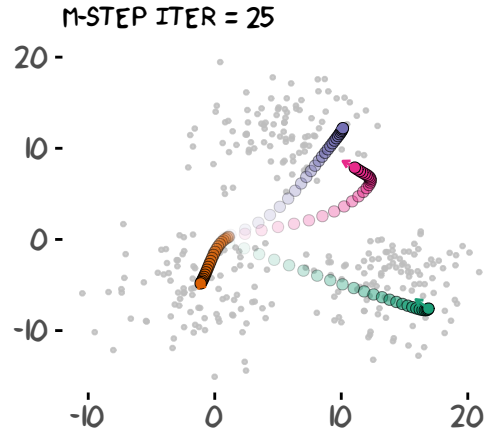
- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



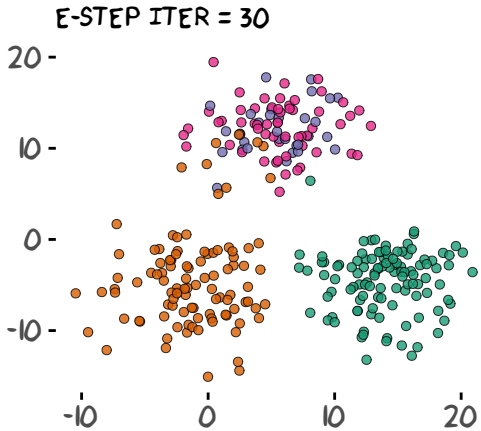


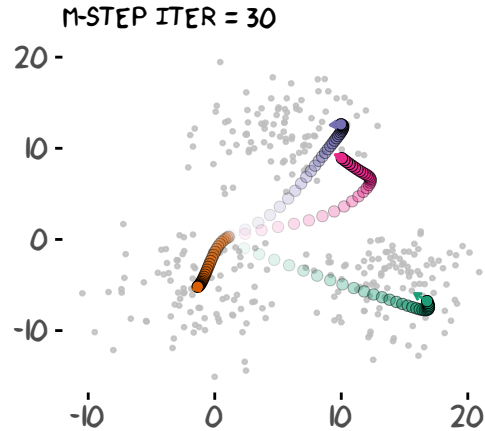
- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



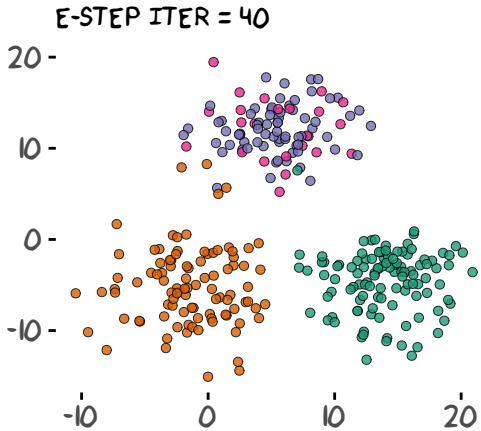


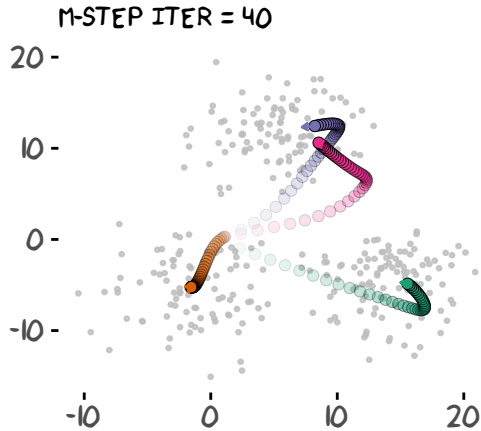
- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



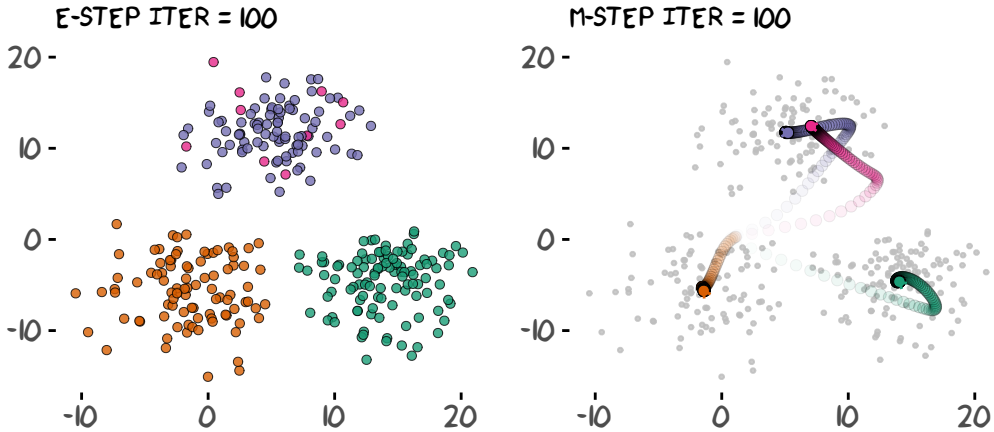


- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

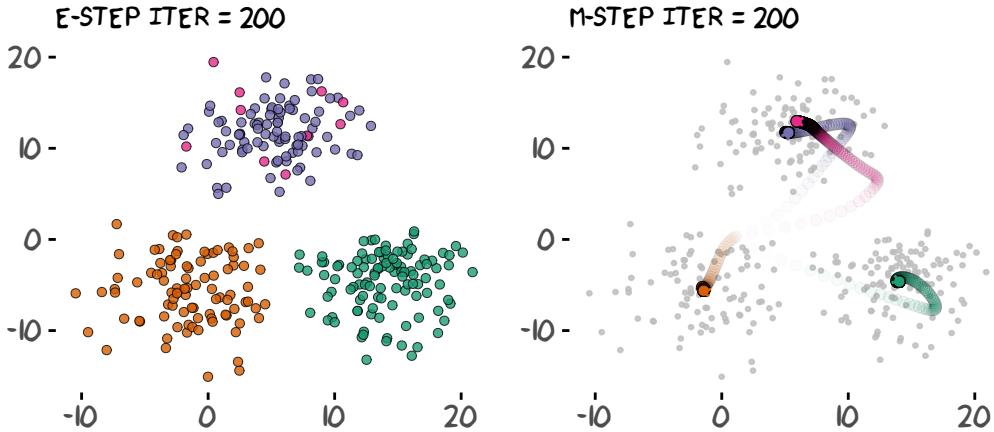




- Arrows: stochastic gradient $\nabla \mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership



- Arrows: stochastic gradient $\nabla\mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

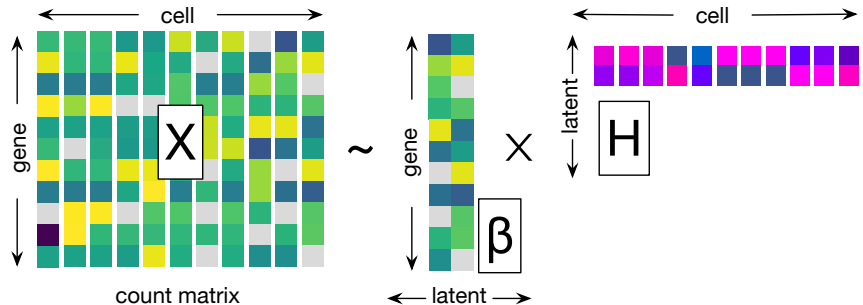


- Arrows: stochastic gradient $\nabla\mu$ (initially $\mu_k = 0$ for all k)
- Colour: latent membership

Today's lecture

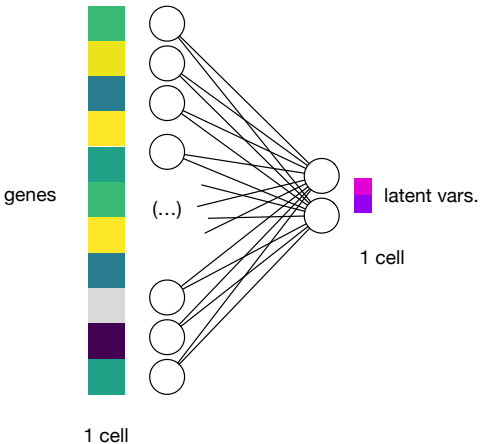
1. Representation learning
2. Warm-up: Clustering is an unsupervised learning
3. VAE: supervise approaches to unsupervised learning
4. Topic modelling in VAE framework

Recap: How can we learn patterns in unsupervised learning from single cell count data?



- Goal: Find the factor-specific gene dictionary β and hidden factor loading H.

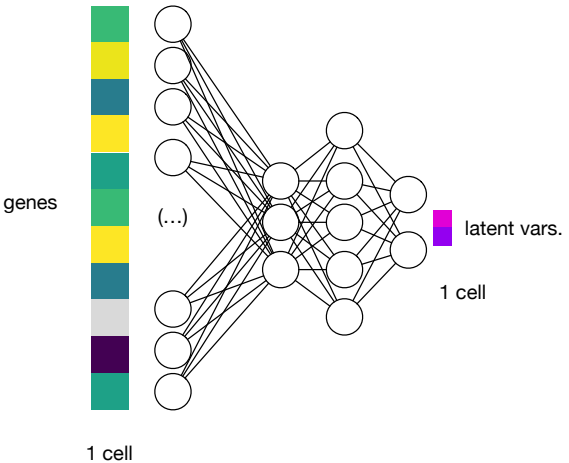
How to “encode” high-dim data to low-dim space?



- Take one cell as a long vector $\mathbf{x}_i$
- Apply an encoding function $f(\mathbf{x}_i)$
- Neural network architectures capture relationships between variables

- no connection within each layer

How to “encode” high-dim data to low-dim space?



- Take one cell as a long vector $\mathbf{x}_i$
- Apply an encoding function $f(\mathbf{x}_i)$
- Neural network architectures capture relationships between variables

$$h_i^{(l)} \leftarrow f\Big(\sum_j \underbrace{W_{ij}}_{\text{connections}} h_j^{(l-1)} + \underbrace{b_i}_{\text{bias}}\Big)$$

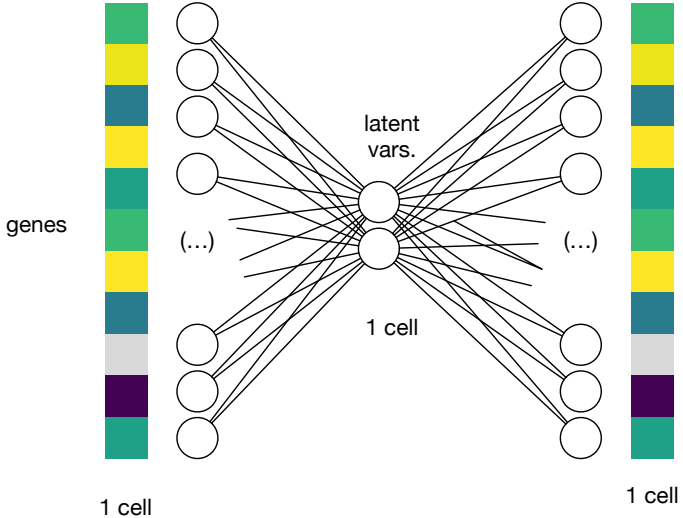
- no connection within each layer

Unsupervised learning of a good encoder model is fundamentally challenging because …

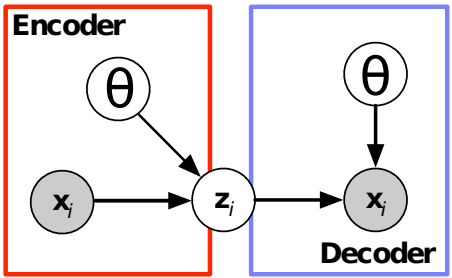
Unsupervised learning of a good encoder model is fundamentally challenging because …

we don’t have good encoding “examples” beforehand.

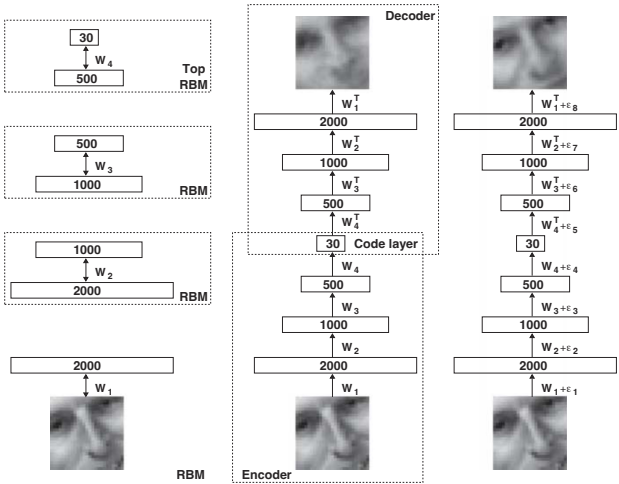
How can we “supervise” how well we encoded?



“Supervise” unsupervised learning by self reconstruction



Deep autoencoder first proposed in computer vision



- Hinton & Salakhutdinov, Science (2006)
- One of the first “deep learning” idea
- Learning the latent representation of an image helps subsequent classification tasks.
- To train a deep autoencoder architecture, they pretrained the model layer by layer.

Let's build a simple autoencoder model using `torch`

```
###################################################################
## R wrapper library for `libtorch` (C++ engine for PyTorch) ##
###################################################################

library(torch)

###########################################
## We can use GPU (NVIDIA Cuda), optional ##
###########################################

GPU <- torch_device("cuda")
```

- See this online book if you are interested in building your own deep learning model: **Deep Learning and Scientific Computing with R torch**

- https://skeydan.github.io/Deep-Learning-and-Scientific-Computing-with-R-torch/

Why do we use some Deep Learning library?

- Built-in functions for scientific computing
  - `log`, `log1p`, `exp`, `softmax`, `sigmoid`, `softplus`
- Faster computation both in CPU and GPU (in general)
- **No need to work on differentiation** by hands
- Trouble shooting by ML community

Encoder: high-dim data $\longrightarrow$ low-dim latent space

```
build.linear.encoder <-
    nn_module(
        ## How do we want to build this module ##
        classname = "linear encoder",
        initialize = function(d.in, K){
            self$K <- K # number of hidden factors
            self$func.z <- nn_linear(d.in, K) # a linear neural net layer
            self$bn <- nn_batch_norm1d(d.in) # batch norm
        },
        ## Define how do we propate infomration ##
        forward = function(x.b){
            self$get.latent(x.b)
        },
        get.latent = function(x.b){
            x.b <- torch_log1p(x.b) # log1p transformation
            x.b <- self$bn(x.b) # to expedite training
            self$func.z(x.b) # take linear combinations
        })
```

Check if this encoder module works okay

```
lin.encoder <- build.linear.encoder(ncol(x.torch), 7)$to(dev = GPU)
lin.encoder
```

```
## An `nn_module` containing 172,267 parameters.
##
## -- Modules ---------------------------------------------------------------------------------------------------
## * func.z: <nn_linear> #133,987 parameters
## * bn: <nn_batch_norm1d> #38,280 parameters
```

```
##########################################
## Test using the first five cells/data ##
##########################################

lin.encoder(x.torch[1:5, ])
```

```
## torch_tensor
## -0.0662 -0.1181  0.9545 -0.3220  0.0896  0.1403  0.1048
##  0.0688 -0.1400 -1.0011  0.8276  0.7960  1.3088  0.2332
##  0.2473 -0.0813  0.4001 -0.0452 -0.5019 -0.7506 -0.3036
## -0.5400  0.2192 -0.0184  0.1981 -0.6727  0.3204  0.1169
##  0.2616  0.0883 -0.3474 -0.6500  0.2994 -0.9912 -0.1629
## [ CUDAFloatType{5,7} ][ grad_fn = <AddmmBackward0> ]
```

Decoder: low-dim latent space $\longrightarrow$ high-dim data

```
build.linear.decoder <-
    nn_module(
        classname = "linear decoder",
        ## Define how we want to build this module
        initialize = function(n.out, K) {
            self$K <- K
            self$func.logX <- nn_linear(K, n.out) # latent dim -> data dim
        },
        ## Define how do get back high-dim data
        forward = function(z.b){
            .num <- self$func.logX(z.b)
            .denom <- torch_logsumexp(.num, dim = -1, keepdim = T)
            return(.num - .denom)
        },
        ## Helper function
        get.weight = function(){
            self$func.logX$weight
        })
```

Check flow from the encoder to decoder

```
lin.decoder <- build.linear.decoder(ncol(x.torch), K=7)$to(device=GPU)

x.input <- x.torch[1:5, ]
z.b <- lin.encoder(x.input)

###############################################
## reconstruction of x based on the latent ##
###############################################
logx.recon <- lin.decoder(z.b)
logx.recon[, 1:7]
```

```
## torch_tensor
## -9.6082 -10.1779 -9.4929 -9.6738 -9.6579 -9.3143 -9.3887
## -10.3584 -9.8389 -10.0204 -9.1980 -9.2597 -10.3298 -9.8388
## -9.8931 -9.8938 -9.5348 -9.5878 -9.9160 -9.3601 -9.7647
## -9.9044 -9.9958 -10.0922 -9.9774 -9.4732 -9.4275 -9.4421
## -10.0828 -9.9529 -9.7000 -9.4871 -9.7266 -9.7873 -10.1206
## [ CUDAFloatType{5,7} ][ grad_fn = <SliceBackward0> ]
```

- Note: the reconstructed data matrix is in logarithm scale

What's really going on in terms of functions?

For each sample/cell i,

- Encoder:

$$\mathbf{z}_i \leftarrow \text{normalize}\big(\log(\mathbf{x}_i + 1)\big) \cdot \mathbf{W}^{(z)} + \mathbf{b}^{(z)}$$

- Decoder:

$$\widehat{\mathbf{x}}_i \leftarrow \mathbf{z}_i \cdot \mathbf{W}^{(x)} + \mathbf{b}^{(x)}$$

Goal: to make the reconstructed data $\approx$ the input

- Gene expression frequency:

$$\rho_{ig} = \frac{\exp(\widehat{\log X}_{ig})}{\sum_{g'} \exp(\widehat{\log X}_{ig'})}$$

- Multinomial log-likelihood:

$$L_i \overset{\text{def}}{=} \sum_{g=1}^{p} X_{ig} \log \rho_{ig}$$

$$\log \rho_{ig} = \widehat{\log X}_{ig} - \log \sum_{g'} \exp(\widehat{\log X}_{ig'})$$

Goal: to make the reconstructed data $\approx$ the input

- Gene expression frequency:

$$\rho_{ig} = \frac{\exp(\widehat{\log X}_{ig})}{\sum_{g'} \exp(\widehat{\log X}_{ig'})}$$

- Multinomial log-likelihood:

$$L_i \overset{\text{def}}{=} \sum_{g=1}^{p} X_{ig} \log \rho_{ig}$$

$$L_i = \sum_{g=1}^{p} X_{ig} \left[\widehat{\log X}_{ig} - \text{logSumExp}(\widehat{\log \mathbf{x}}_i) \right]$$

Goal: to make the reconstructed data $\approx$ the input

```
multinom.llik <- function(x.input, logx.recon){
    torch_sum(x.input * logx.recon, dim = -1)
}

multinom.llik(x.input, logx.recon)
```

```
## torch_tensor
## 1e+05 *
## -2.3280
## -3.1909
## -1.7181
## -3.3844
## -1.6027
## [ CUDAFloatType{5} ][ grad_fn = <SumBackward1> ]
```

- We need to maximize this with respect to all the parameters in the encoder and decoder layers.
- Maximizing log-likelihood $\iff$ minimizing *negative* log-likelihood

`Torch` provides a convenient way to “differentiate”

```
loss <- -torch_mean(multinom.llik(x.input, logx.recon))
loss
```

```
## torch_tensor
## 244483
## [ CUDAFloatType{} ][ grad_fn = <SubBackward0> ]
```

```
loss$backward()
```

Put the encoder and decoder together (so that gradients flow)

```
build.linear.autoenc <-
    nn_module(
        classname = "linear autoencoder",
        initialize = function(d.data, K){
            self$enc <- build.linear.encoder(d.data, K)
            self$dec <- build.linear.decoder(d.data, K)
        },
        forward = function(x){
            z <- self$enc(x)
            x.hat <- self$dec(z)
            .loss <- - multinom.llik(x, x.hat)
            list(loss = .loss)
        })
```

A bit more formal definition

Minimize the total loss

$$L \overset{\text{def}}{=} \sum_{i=1}^{n} \text{Loss}(\mathbf{x}_i, \widehat{\mathbf{x}}_i)$$

where

$$\widehat{\mathbf{x}}_i = \text{Decoder}\big(\text{Encoder}\big(\mathbf{x}_i; \theta^{(\text{enc})}\big); \theta^{(\text{dec})}\big)$$

with respect to the parameters θ.

Update by stochastic gradient steps

Take gradient (direction to minimize the loss) for each parameter:

$$\nabla L \equiv \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \ldots \right)$$

Update parameters:

$$\theta^{(t)} \leftarrow \underbrace{\theta^{(t-1)}}_{\text{the previous}} - \underbrace{\rho_t}_{\text{learning rate}} \underbrace{\nabla L}_{\text{gradient at } t-1}$$

Update by stochastic gradient steps

```
## register parameters to ADAM optimizer
.params <- linear.autoenc$parameters
adam <- optim_adam(.params, lr = 1e-2)
adam$zero_grad()
```

```
x.b <- x.torch[1:3, ]       # Select minibatch data
out <- linear.autoenc(x.b) # data -> latent -> recon.
out$loss                    # loss evalulated on X.b
```

```
## torch_tensor
## 1e+05 *
##  2.3274
##  3.1810
##  1.7039
## [ CUDAFloatType{3} ][ grad_fn = <SubBackward0> ]
```

1. Take minibatch $X^{(b)}$
2. Recon. $\hat{X}^{(b)} = \text{Dec}\big(\text{Enc}\big(X^{(b)}\big)\big)$
3. Evaluate $\text{Loss}\big(X^{(b)}, \hat{X}^{(b)}\big)$

Update by stochastic gradient steps

```
loss.b <- torch_sum(out$loss) #
loss.b$backward()             # numerically
                              # differentiate

## Before we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
##  1.3499
## -0.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

```
adam$step() -> .
## After we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
## -8.6501
## -10.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

Update by stochastic gradient steps

```
loss.b <- torch_sum(out$loss) #
loss.b$backward()             # numerically
                              # differentiate

## Before we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
##  1.3499
## -0.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

```
adam$step() -> .
## After we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
## -8.6501
## -10.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

⋆ Aggregate training loss across samples in this minibatch: $\sum_{i \in \text{minibatch}(b)} \text{loss}\left(\mathbf{x}_i, \widehat{\mathbf{x}_i}\right)$

Update by stochastic gradient steps

```
loss.b <- torch_sum(out$loss) #
loss.b$backward()             # numerically
                              # differentiate

## Before we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
##  1.3499
## -0.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

```
adam$step() -> .
## After we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
## -8.6501
## -10.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

⋆ Aggregate training loss across samples in this minibatch: $\sum_{i \in \text{minibatch}(b)} \text{loss}\left(\mathbf{x}_i, \widehat{\mathbf{x}_i}\right)$

⋆ Take gradient with respect to encoder and decoder parameters

Update by stochastic gradient steps

```
loss.b <- torch_sum(out$loss) #
loss.b$backward()             # numerically
                              # differentiate

## Before we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
##  1.3499
## -0.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

```
adam$step() -> .
## After we take one SGD step
head(adam$param_groups[[1]]$params$enc.func.z.bias, 2)
```

```
## torch_tensor
## 0.001 *
## -8.6501
## -10.5025
## [ CUDAFloatType{2} ][ grad_fn = <IndexBackward0> ]
```

⋆ Aggregate training loss across samples in this minibatch:
$\sum_{i \in \text{minibatch}(b)} \text{loss}(\mathbf{x}_i, \widehat{\mathbf{x}}_i)$

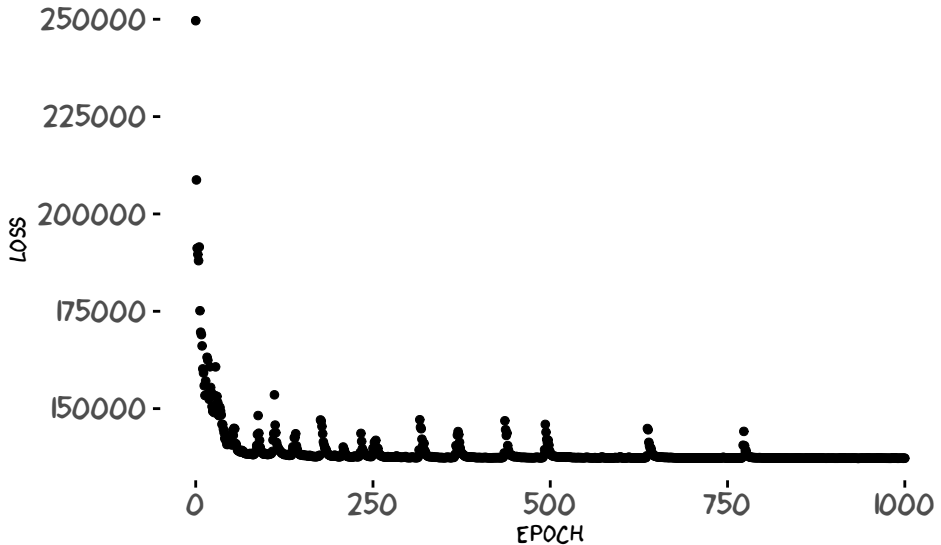
⋆ Take gradient with respect to encoder and decoder parameters

⋆ Update the parameters by taking one (stochastic) gradient descent step
$\theta^{(t)} \leftarrow \theta^{(t-1)} + \rho \nabla L$

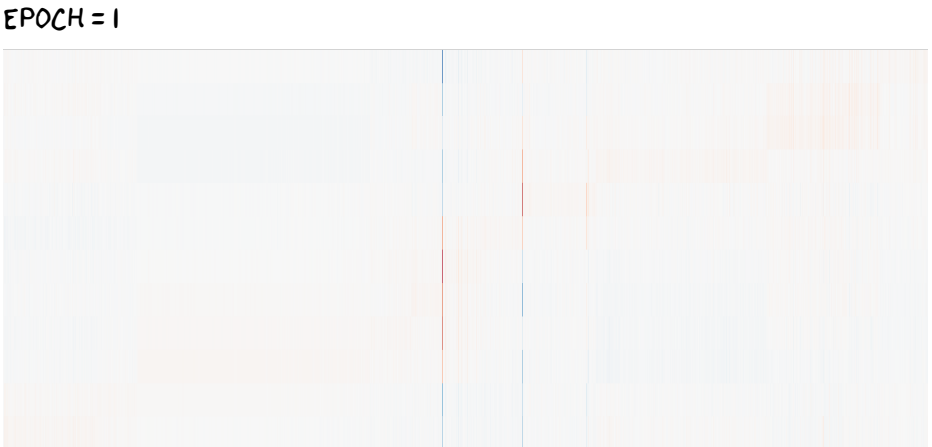
Training algorithm: Repeat SGD steps many times

- For many epochs
  1. Sample mini batch data
  2. Evaluate loss function $L(\mathbf{x}_i, \hat{\mathbf{x}}_i)$
  3. Compute gradient $\nabla_\theta L$
  4. Update parameters by SGD
- Report encoding results

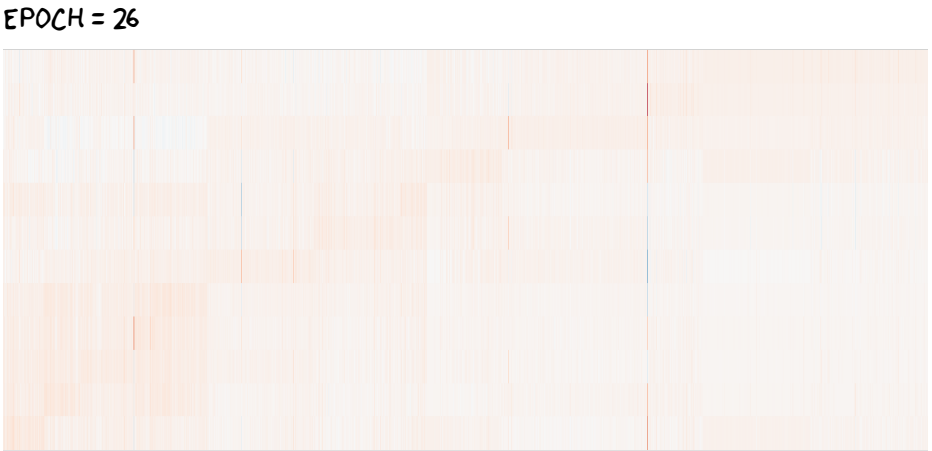
Results: SGD minimized the loss function



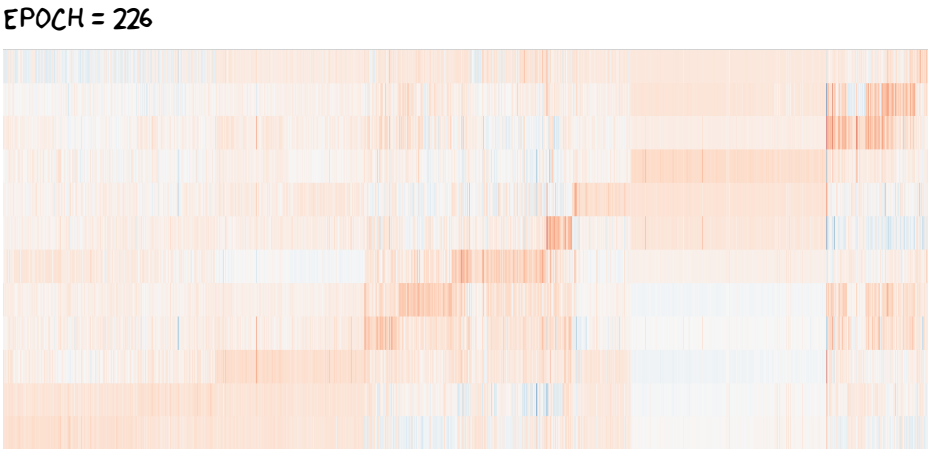
Hidden representations (factor × cell)



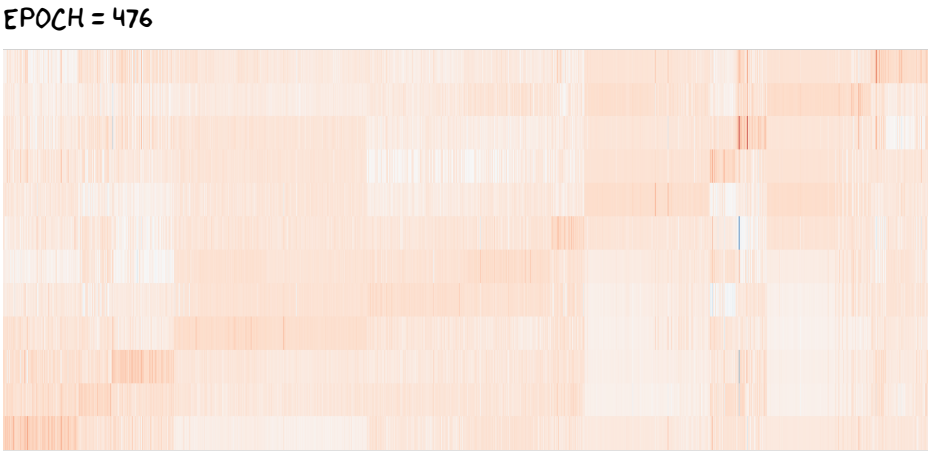
Hidden representations (factor × cell)



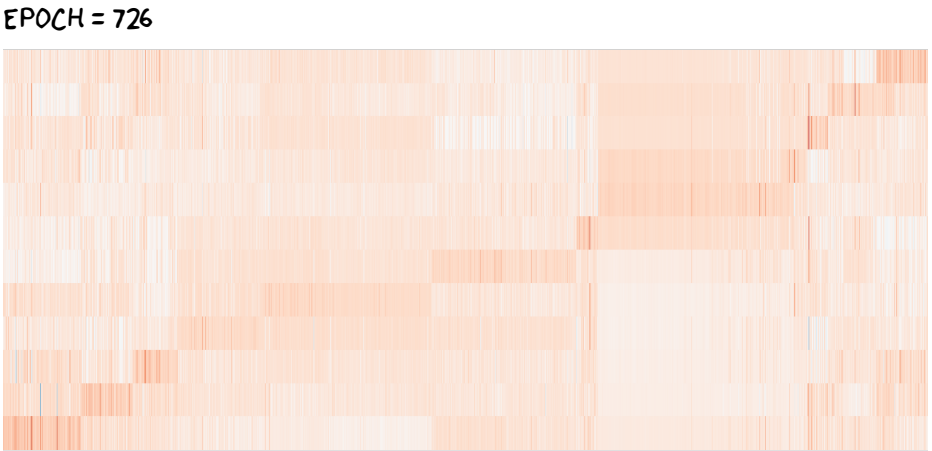
Hidden representations (factor × cell)



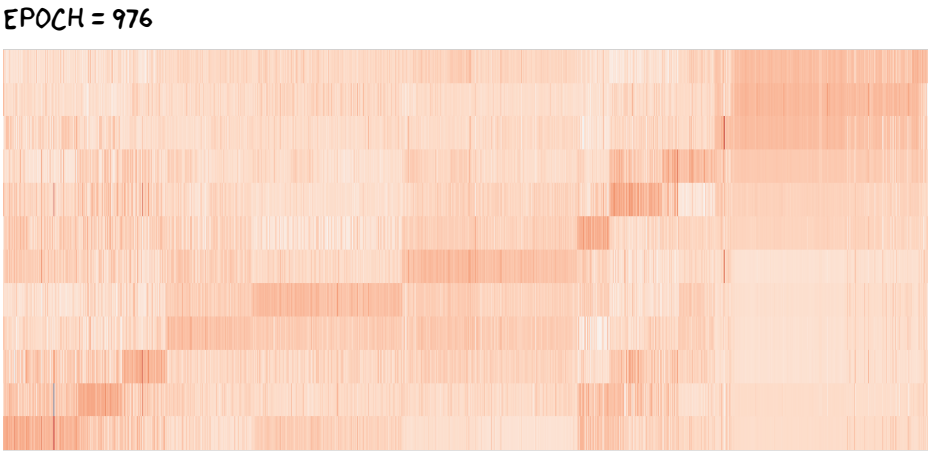
Hidden representations (factor × cell)



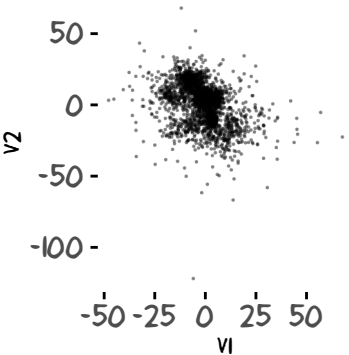
Hidden representations (factor × cell)

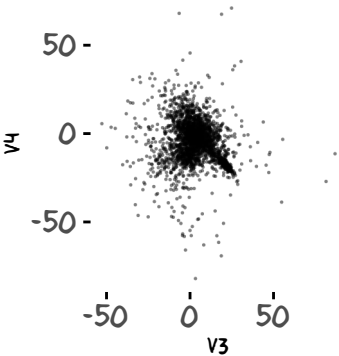


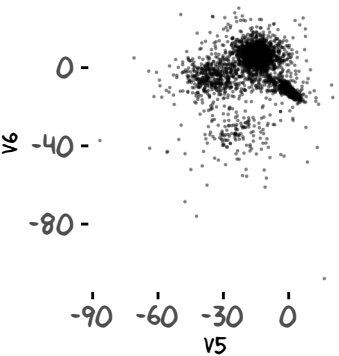
Hidden representations (factor × cell)



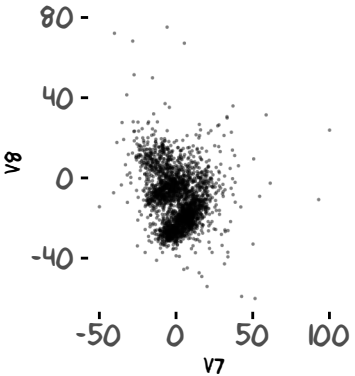
Latent dimensions estimated by the encoder model

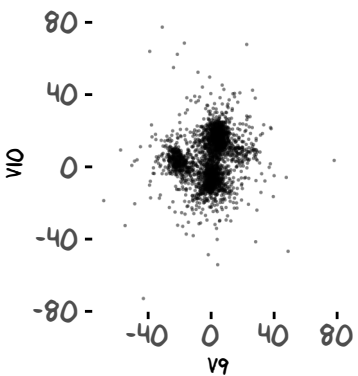


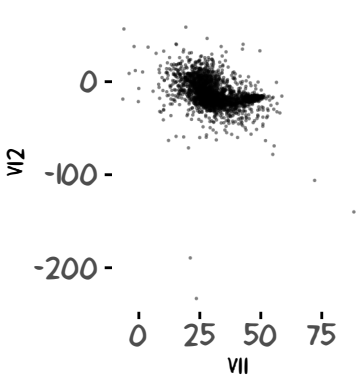




Latent dimensions estimated by the encoder model

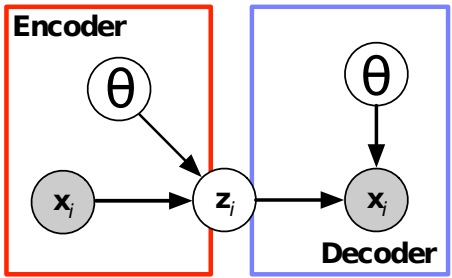




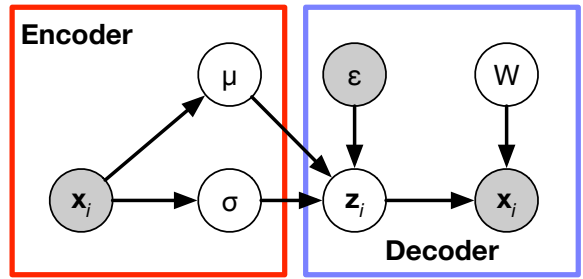


Variational autoencoder (VAE)

A classical autoencoder:



Variational autoencoder:



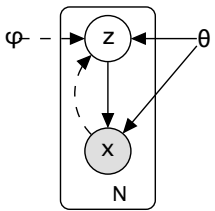
- Define relationships between variables (auto generative process)
- Usually, the decoder side captures our scientific hypothesis

VAE approximates Bayesian inference

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com



Prior:

$$Z \sim p(Z|\psi)$$

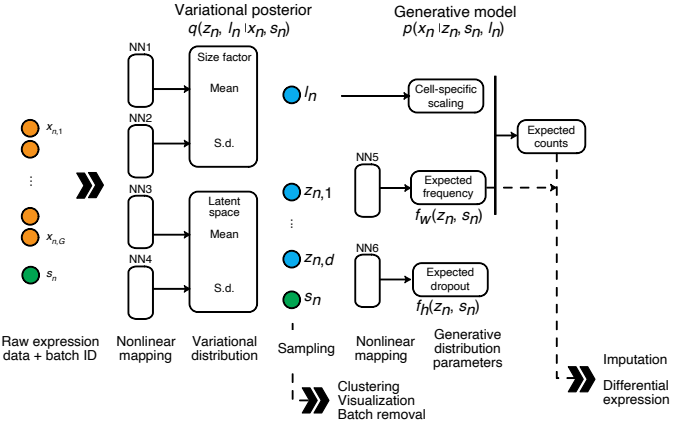
Data likelihood:

$$X \sim p(X|Z,\theta)$$

Variational Encoder:

$$q(Z|X) = \mathsf{NN}(X)$$

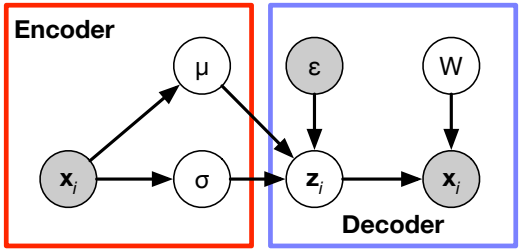
SCVI: deep generative model for scRNA-seq



Generative model: zero-inflated negative binomial distribution

Lopez, .., Jordan, Yosef, *Nature Methods* (2018)

A new encoder as a posterior inference machine



We need to define functions (neural networks) that maps from the data vector **x** to

- the mean of latent embedding: μ
- the log variance of latent embedding: σ
- $z_{ig} \leftarrow \mu_{ig} + \sigma_{ig}\epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$

A new encoder as a posterior inference machine

```
build.vae.encoder <- nn_module(
        classname = "vae encoder",
        initialize = function(d.in, K){
            self$K <- K                                          # number of hidden vars.
            self$z.mean <- nn_linear(d.in, K)                    # mean function
            self$z.logvar <- nn_linear(d.in, K)                  # log variance function
            self$bn <- nn_batch_norm1d(d.in)                     # batch norm
        },
        forward = function(x.b){
            x.b <- self$normalize(x.b)
            mm <- self$z.mean(x.b)                               # mean evaluated
            lv <- torch_clamp(self$z.logvar(x.b), -4.0, 4.0)     # log-var evaluated
            z <- mm + torch_randn_like(lv) * torch_exp(lv/ 2.) # stochastic z
            list(z = z, z.mean = mm, z.logvar = lv)
        },
        normalize = function(x.b){                               # normalization
            x.b <- torch_log1p(x.b)                              # log1p transformation
            self$bn(x.b)                                         # to expedite training
        }, ## helper function
        get.latent = function(x.b){ self$z.mean(self$normalize(x.b)) })
```

We will have two types of loss functions

Data log likelihood (a generative model)

$$\log \prod_{i=1}^{n} p(\mathbf{x}_i|\mathbf{z}_i)$$

multinomial

```
multinom.llik <- function(x.input, logx.recon){
    torch_sum(x.input * logx.recon, dim = -1)
}
```

- The log-likelihood is the same as before
- KL loss will work like regularization

Divergence between prior and posterior

$$D_{\mathrm{KL}}\left(q(\mathbf{z}) \middle\| p(\mathbf{z})\right)$$

posterior prior

```
kl.loss <- function(.mean, .lnvar) {
    -0.5 * torch_sum(1. + .lnvar
                     - torch_pow(.mean, 2.)
                     - torch_exp(.lnvar),
                     dim = -1)
}
```

- We assume both q and p follows Gaussian distribution

A complete definition of our VAE model

```
build.vae <-
    nn_module(
        classname = "variational autoencoder",
        initialize = function(d.data, K){
            self$enc <- build.vae.encoder(d.data, K)     # encoder model
            self$dec <- build.linear.decoder(d.data, K) # decoder model
        },
        forward = function(x){
            .enc <- self$enc(x)                          #
            x.hat <- self$dec(.enc$z)                    # reconstruction
            .llik <- multinom.llik(x, x.hat)             # data likelihood
            .kl <- kl.loss(.enc$z.mean, .enc$z.logvar)   # KL divergence
            .loss <- .kl - .llik                         # combined loss
            list(loss = .loss, kl=.kl)
        }
    )
```

- What do you want to change?

VAE: Check flow from the encoder to decoder

```
vae <- build.vae(ncol(x.torch), K=12)
vae$to(device = GPU)

x.input <- x.torch[1:5, ]
out <- vae(x.input)
out$loss
```

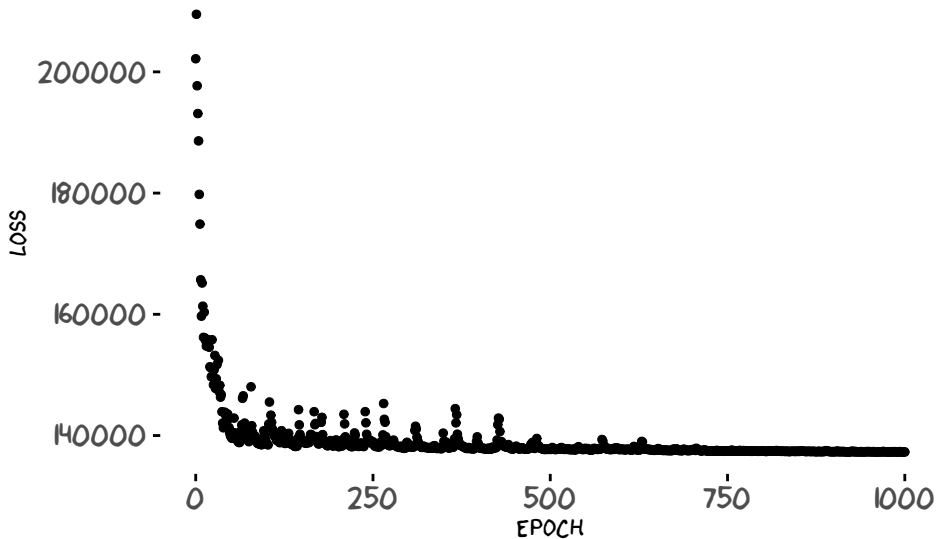
```
## torch_tensor
## 1e+05 *
##  2.3652
##  3.2222
##  1.7579
##  3.5041
##  1.6422
## [ CUDAFloatType{5} ][ grad_fn = <SubBackward0> ]
```

```
##############################################
## reconstruction of x based on the latent ##
##############################################
z.b <- vae$enc(x.input)
logx.recon <- vae$dec(z.b$z)
logx.recon[, 1:5]
```

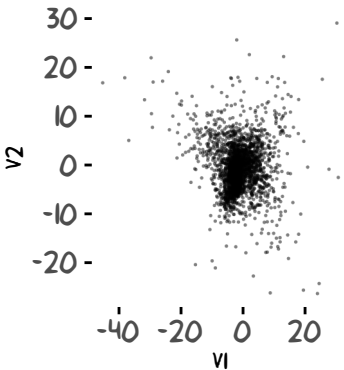
```
## torch_tensor
## -9.8588 -8.8405 -9.7515 -10.7885 -9.7036
## -11.7461 -11.2437 -9.9606 -9.3373 -11.8871
## -10.3910 -10.4692 -9.5067 -10.1803 -10.3123
## -11.0614 -10.1565 -10.2433 -10.3422 -10.7016
## -10.3482 -10.4923 -11.3923 -8.2034 -11.2555
## [ CUDAFloatType{5,5} ][ grad_fn = <SliceBackward0>
```

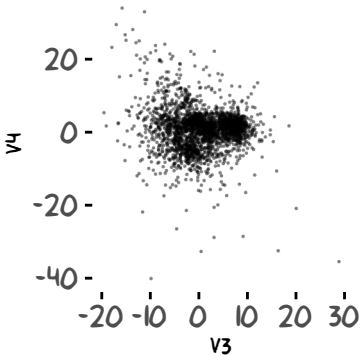
- Note: the reconstructed data matrix is in logarithm scale

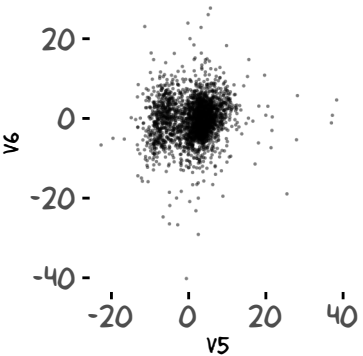
Results: SGD minimized the VAE loss function



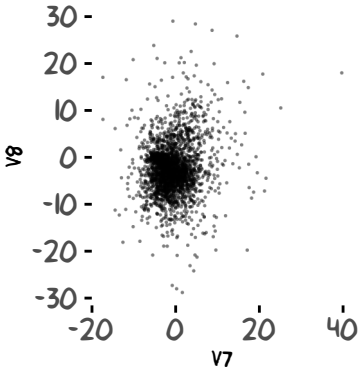
Latent dimensions in the VAE model

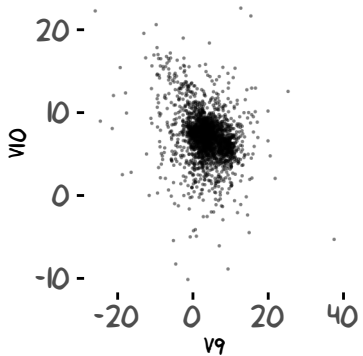


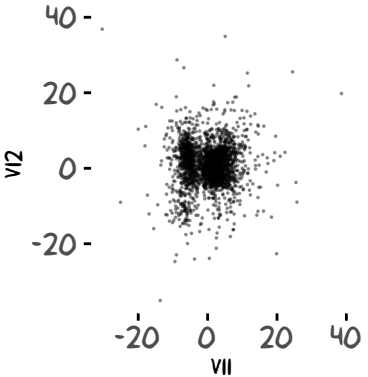




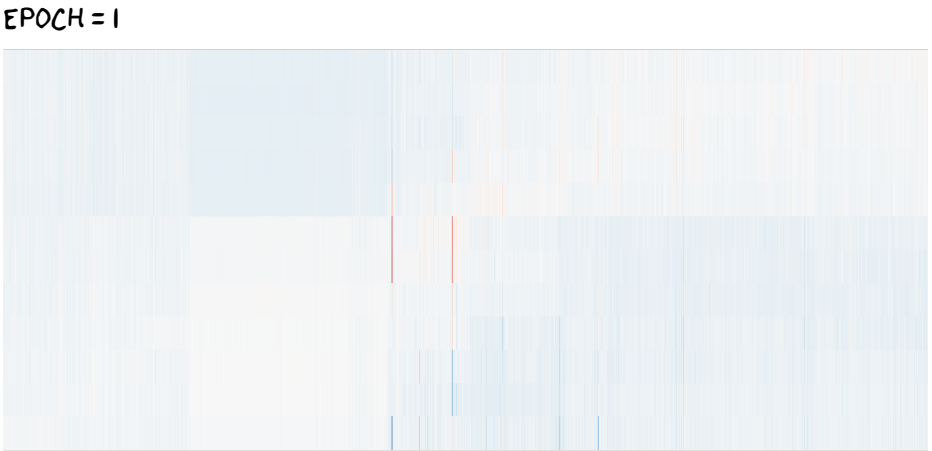
Latent dimensions in the VAE model





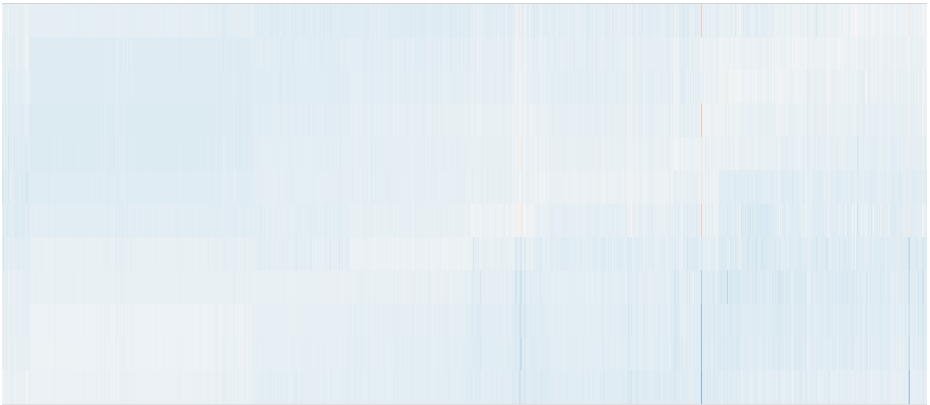


Hidden representations (factor × cell)

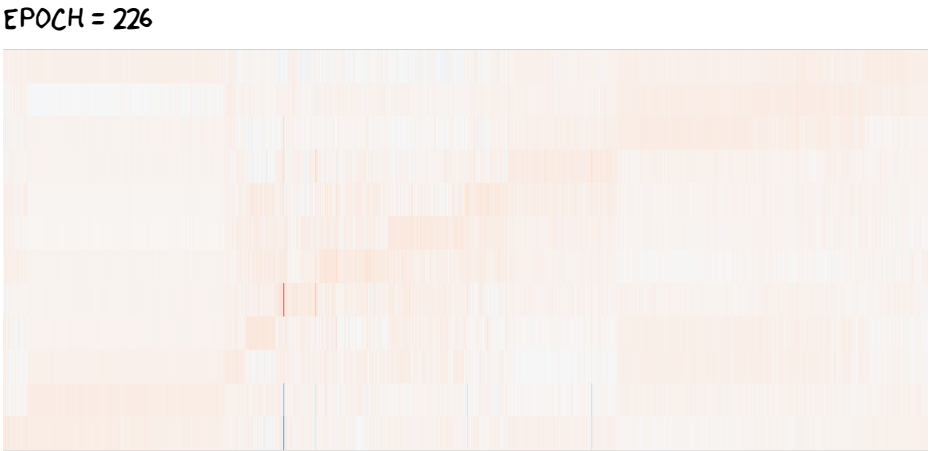


Hidden representations (factor × cell)

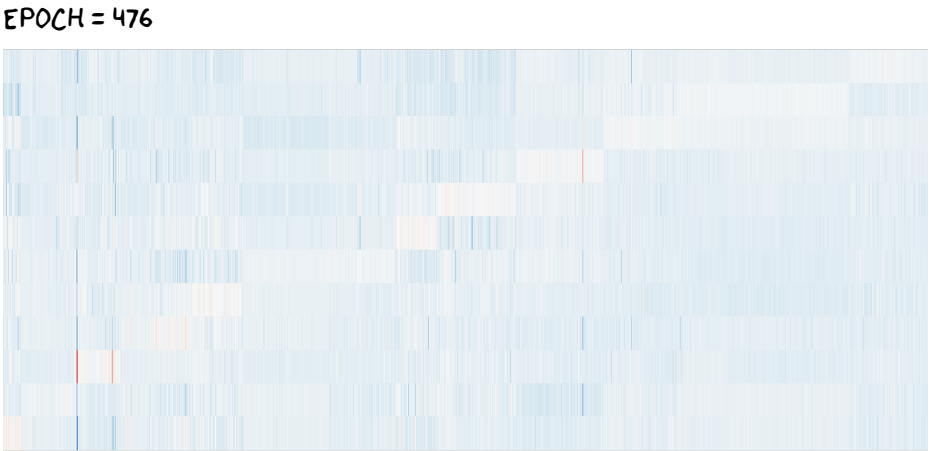
EPOCH = 26



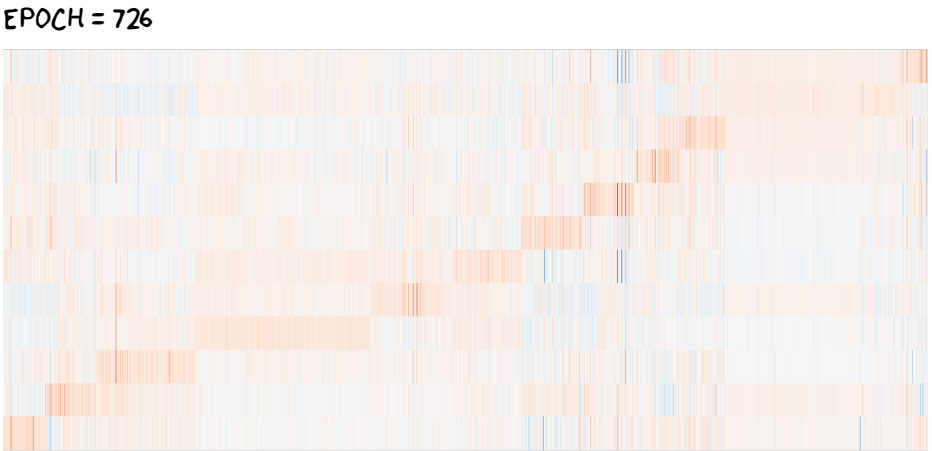
Hidden representations (factor × cell)



Hidden representations (factor × cell)

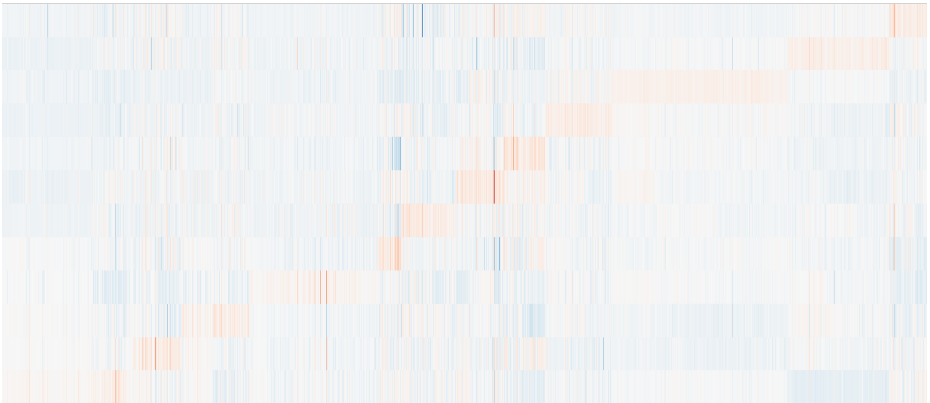


Hidden representations (factor × cell)

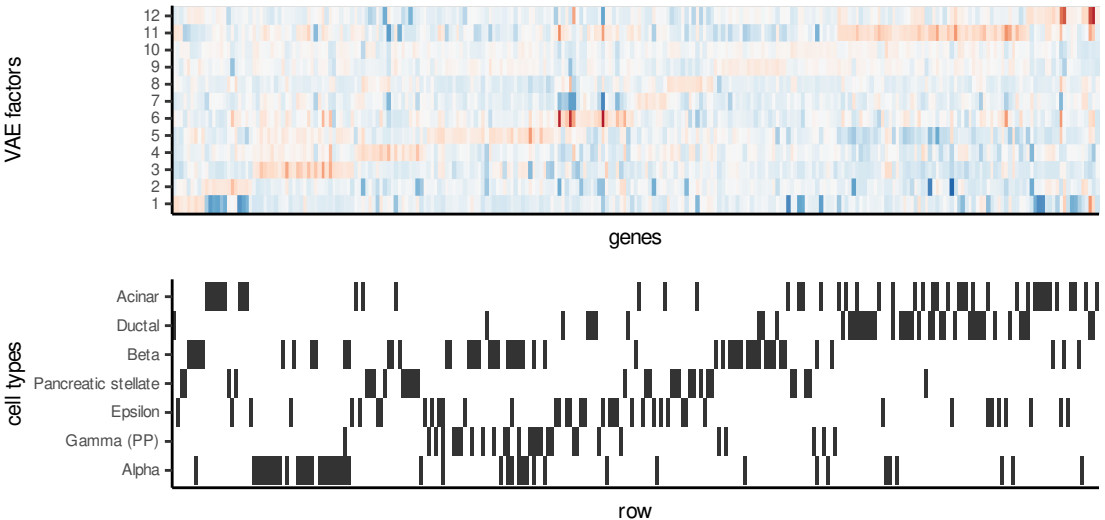


Hidden representations (factor × cell)

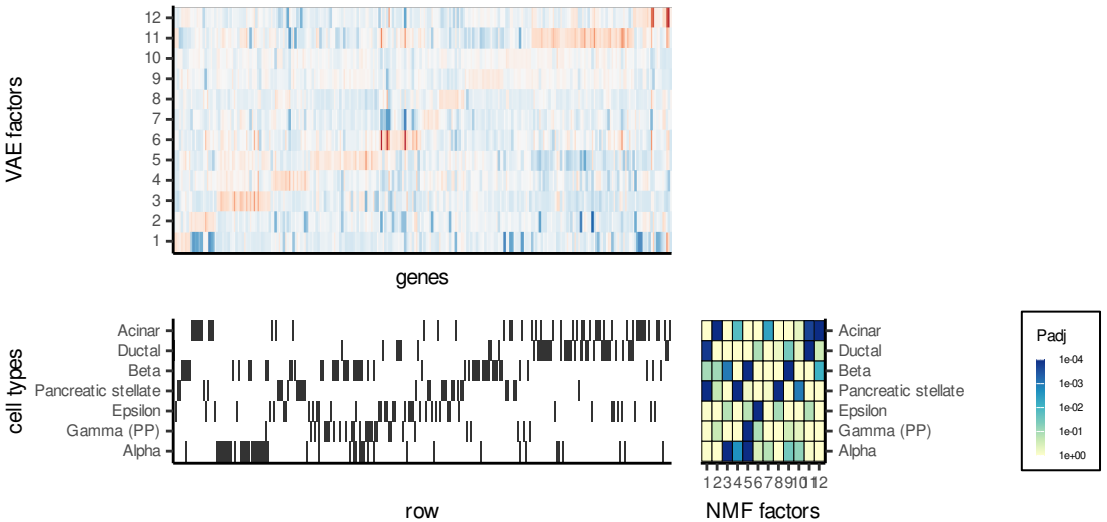
EPOCH = 976



Annotate factors to cell types by enrichment (`fgsea`)



Annotate factors to cell types by enrichment (`fgsea`)



Today's lecture

1. Representation learning
2. Warm-up: Clustering is an unsupervised learning
3. VAE: supervise approaches to unsupervised learning
4. Topic modelling in VAE framework

Can we improve model interpretation?

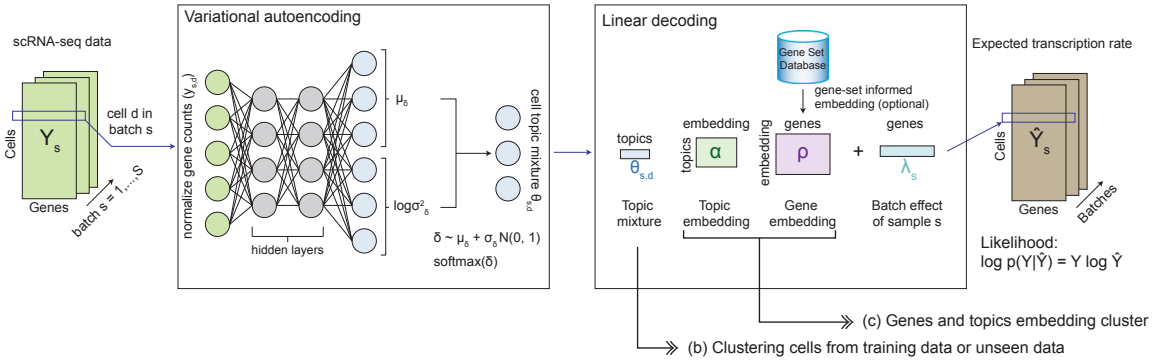
Can we improve model interpretation?

1. The decoder part is open to modelling in many different ways.

Can we improve model interpretation?

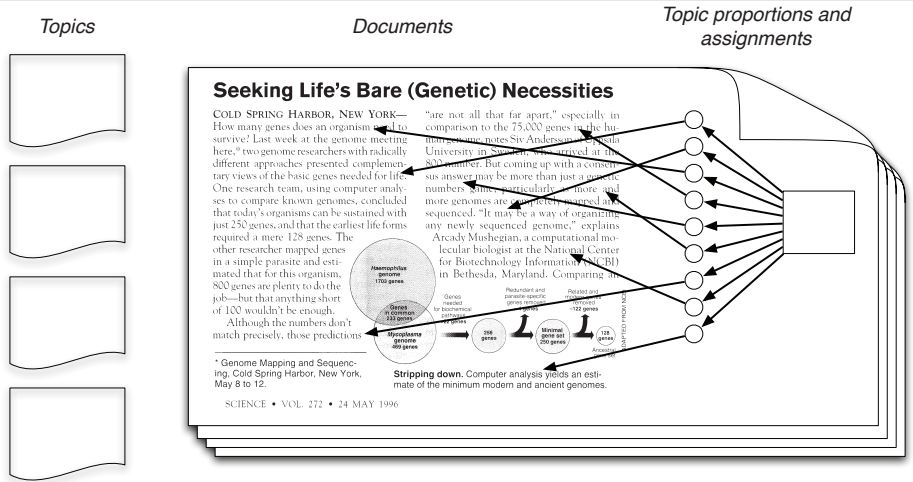
1. The decoder part is open to modelling in many different ways.
2. Can we define latent factors by gene expression frequencies?

Single-cell Embedded Topic Model



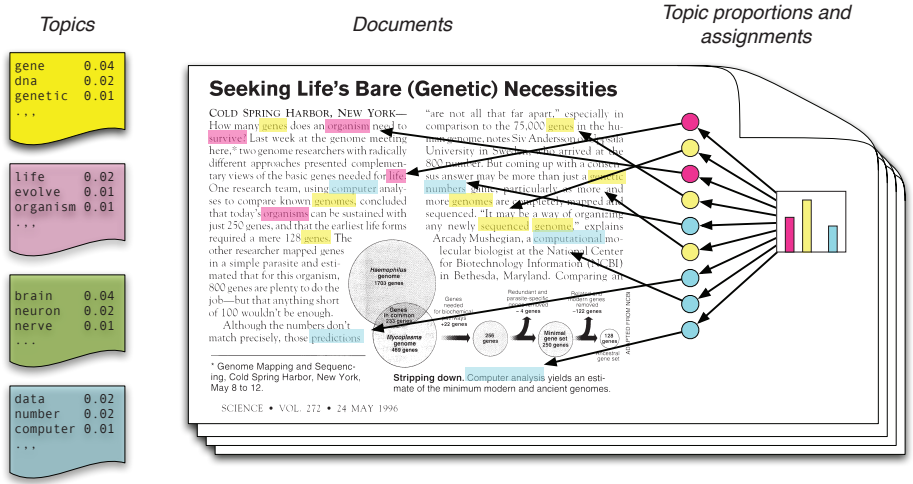
Zhao .. Li, Nature Comm. (2021)

Document topic modelling



Slide credit: David Blei

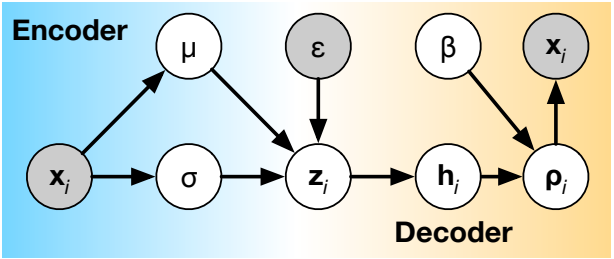
Word frequencies define topics in documents



Slide credit: David Blei

Multinomial topic model for scRNA-seq data

Can we simply model scRNA-seq counts by multinomial distribution?



- X_{ig}: gene expression of a gene g in a single cell i
- H_{ik}: latent topic proportion of a cell i to a topic k
- β_{kg}: topic k-specific gene probability

Multinomial topic model for scRNA-seq data

Can we simply model scRNA-seq counts by multinomial distribution?

Likelihood model:

$$\mathcal{L} = \prod_{i=1}^{n} \prod_{g=1}^{\text{genes}} \left(\sum_{k} H_{ik} \beta_{kg} \right)^{X_{ij}}$$

- X_{ig}: gene expression of a gene g in a single cell i
- H_{ik}: latent topic proportion of a cell i to a topic k
- β_{kg}: topic k-specific gene probability

Multinomial topic model for scRNA-seq data

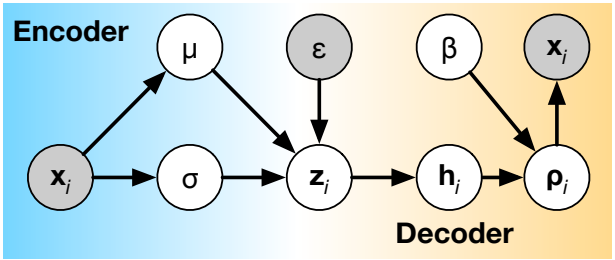
Can we simply model scRNA-seq counts by multinomial distribution?

Likelihood model:

$$\mathcal{L} = \prod_{i=1}^{n} \prod_{g=1}^{\text{genes}} \left(\underbrace{\sum_{k} H_{ik}\beta_{kg}}_{\text{a gene } g\text{'s probability in a cell } i \equiv \rho_{ig}} \right)^{X_{ij}}$$

- X_{ig}: gene expression of a gene g in a single cell i
- H_{ik}: latent topic proportion of a cell i to a topic k
- β_{kg}: topic k-specific gene probability

Topic modelling for single-cell data



Probability of gene g in a cell i:

$$\rho_{ig} = \sum_{k \in \text{topics}} H_{ik} \beta_{kg}$$

By **not** normalizing the probability of each cell, we do not worry about modelling sequencing depths.

Document topic modelling vs. single-cell ETM

variables	in document topic model	in single cell ETM
D	Total number of documents (corpus)	Total number of cells
d	Document index	Cell index
N_d	Number of words in a document d	Number of read counts in a cell d
j	Word index, $j \in [N_d]$	Read index
K	Total number of topics	Total number of cell type topics
k	Topic index, $k \in [K]$	Cell topic index
V	Size of vocabulary	Total number of genes
v	Vocabulary index $v \in [V]$	Gene index
W_{dj}^v	Indicator for a word to vocabulary $\in \{0, 1\}$	Indicator for a read to a gene $\in \{0, 1\}$
X_{dv}	Vocabulary v occurrence in a document d	Gene expression of a gene v in a cell $d \in [0, N_d]$

variables	in document topic model	in single cell ETM
Z_{dj}^k	Indicator for assigning a word to a topic k	Indicator for assigning a read to a topic k
H_{dk}	Hidden state k of a document d	Hidden state k of a cell d
β_{kv}	topic k-specific vocabulary v frequency	topic k-specific, a gene v's exression

- In Latent Dirichlet Allocation: $\sum_{k=1}^{K} H_{dk} = 1$ and $H_{dk} > 0$, and $\mathbf{h}_d \sim \text{Dirichlet}(\alpha/K, \dots, \alpha/K)$ *a priori*. Approximately, we have $\hat{H}_{dk} = \sum_{j}^{N_d} Z_{dj}^k / N_d$.
- In Embedded Topic model, H_{dk} with the simplex constraints; $H_{dk} = \exp(\delta_{dk}) / \sum_{k'} \exp(\delta_{dk'})$ where $\delta_{dk} \sim \mathcal{N}(0, 1)$ *a priori*.
- Additional constraints: $\beta_{kv} > 0$ and $\sum_v \beta_{kv} = 1$, meaning that only a handful of vocabulary v contribute to a topic k.

Let's modify the decoder part

```
build.etm.decoder <-
    nn_module(classname = "ETM decoder",
        initialize = function(n.out, K, jitter = 1e-2) {
            self$lbeta <- nn_parameter(torch_randn(K, n.out) * jitter)
            self$beta <- nn_log_softmax(2) # topic x variant (softmax for each topic)
            self$hid <- nn_log_softmax(2)  # sample x topic (softmax for each sample)
        },
        ## Define how do get back high-dim data
        forward = function(z.b, eps = 1e-8){
            .beta <- self$get.weight()
            h.b <- self$hid(z.b)
            torch_log(torch_mm(torch_exp(h.b), torch_exp(.beta)) + eps)
        },
        ## Helper function
        get.weight = function(){
            self$beta(self$lbeta)
        })
```

ETM: putting the encoder and decoder together

```
build.etm <-
    nn_module(
        classname = "embedded topic model",
        initialize = function(d.data, K){
            self$enc <- build.vae.encoder(d.data, K)
            self$dec <- build.etm.decoder(d.data, K)
        },
        forward = function(x){
            .enc <- self$enc(x)
            x.hat <- self$dec(.enc$z)
            .llik <- multinom.llik(x, x.hat)
            .kl <- kl.loss(.enc$z.mean, .enc$z.logvar)
            .loss <- .kl - .llik
            list(loss = .loss, kl = .kl, latent = .enc$z.mean)
        })
```

ETM: Check flow from the encoder to decoder

```
etm <- build.etm(ncol(x.torch), K=12)
etm$to(device = GPU)

x.input <- x.torch[1:5, ]
out <- etm(x.input)

out <- vae(x.input)
out$loss
```

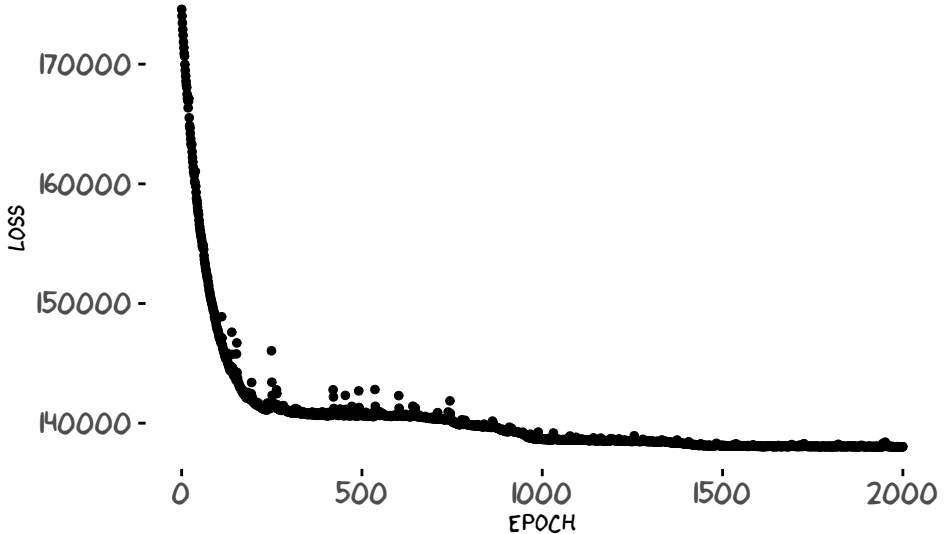
```
## torch_tensor
## 1e+05 *
##  2.3500
##  3.2781
##  1.7583
##  3.4109
##  1.6215
## [ CUDAFloatType{5} ][ grad_fn = <SubBackward0> ]
```

```
##############################################
## reconstruction of x based on the latent ##
##############################################
z.b <- etm$enc(x.input)
logx.recon <- etm$dec(z.b$z)
logx.recon[, 1:5]
```

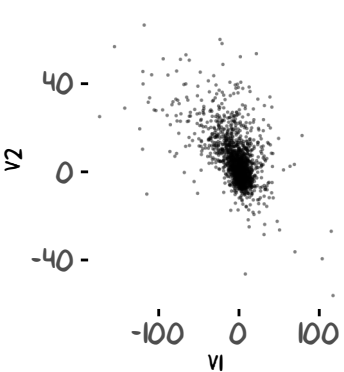
```
## torch_tensor
## -9.8548 -9.8533 -9.8600 -9.8523 -9.8643
## -9.8556 -9.8536 -9.8587 -9.8546 -9.8627
## -9.8598 -9.8446 -9.8603 -9.8621 -9.8601
## -9.8599 -9.8491 -9.8553 -9.8594 -9.8579
## -9.8558 -9.8571 -9.8601 -9.8513 -9.8635
## [ CUDAFloatType{5,5} ][ grad_fn = <SliceBackward0>
```

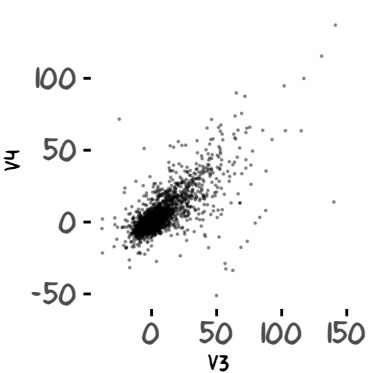
- Note: the reconstructed data matrix is in logarithm scale

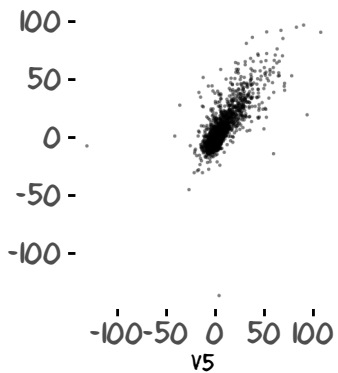
Results: SGD minimized the loss function



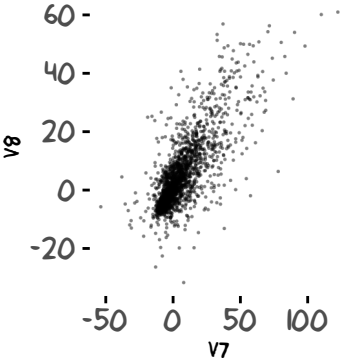
Latent dimensions in the ETM model

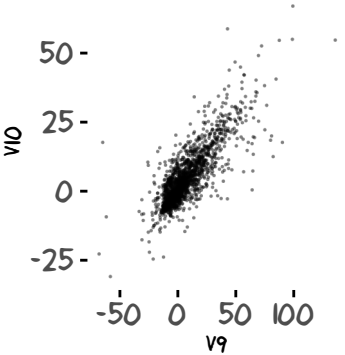


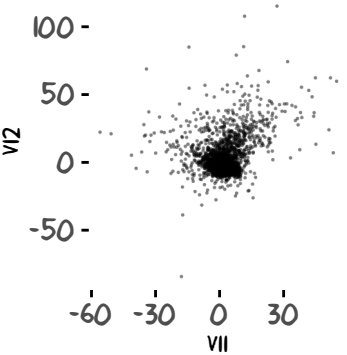




Latent dimensions in the ETM model



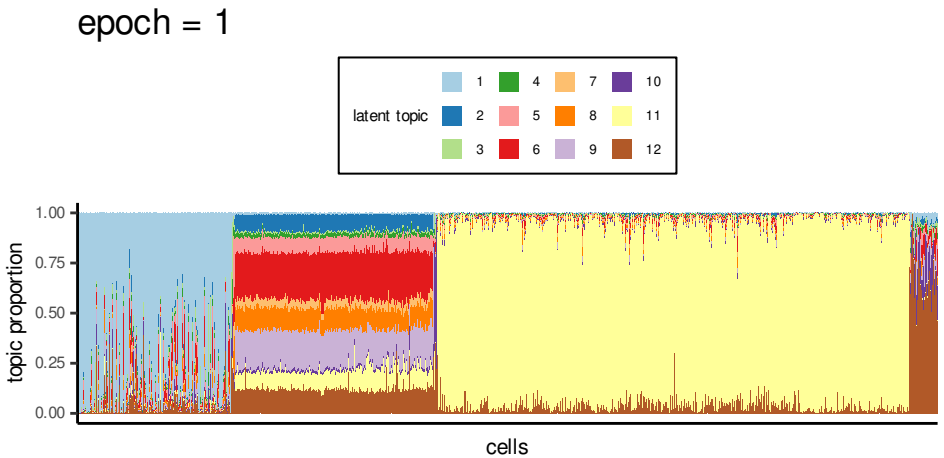




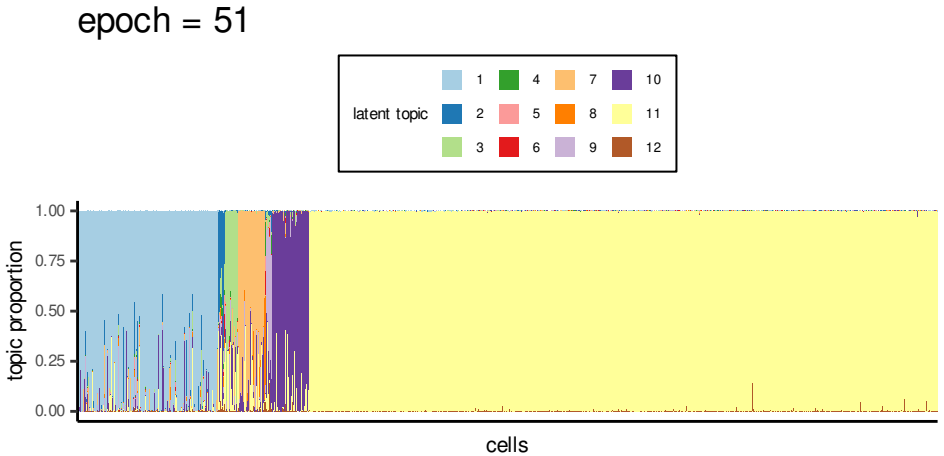
In ETM, the hidden dimensions are not independent

$$H_{ik} = \frac{\exp(Z_{ik})}{\sum_{k'} \exp(Z_{ik'})}$$

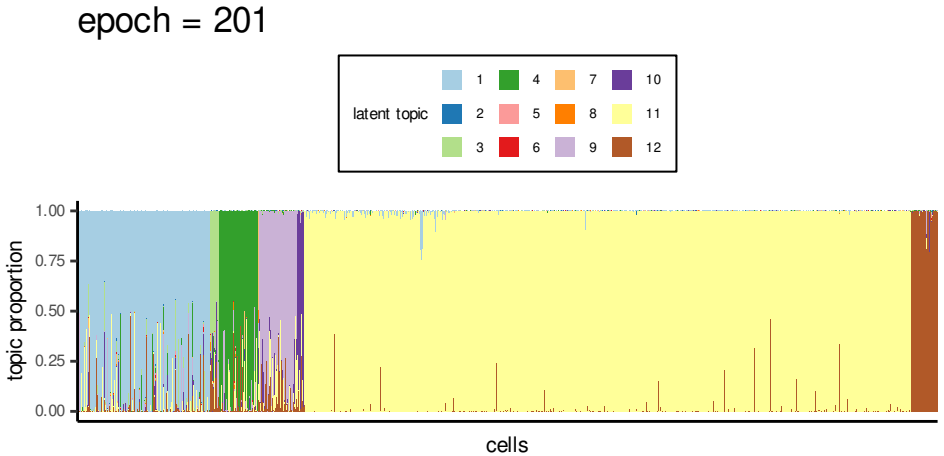
Hidden representations (mixing proportions)



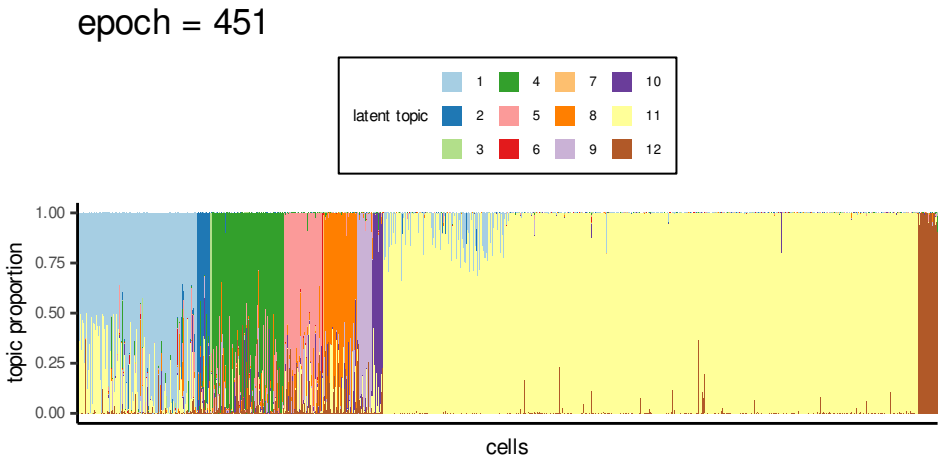
Hidden representations (mixing proportions)



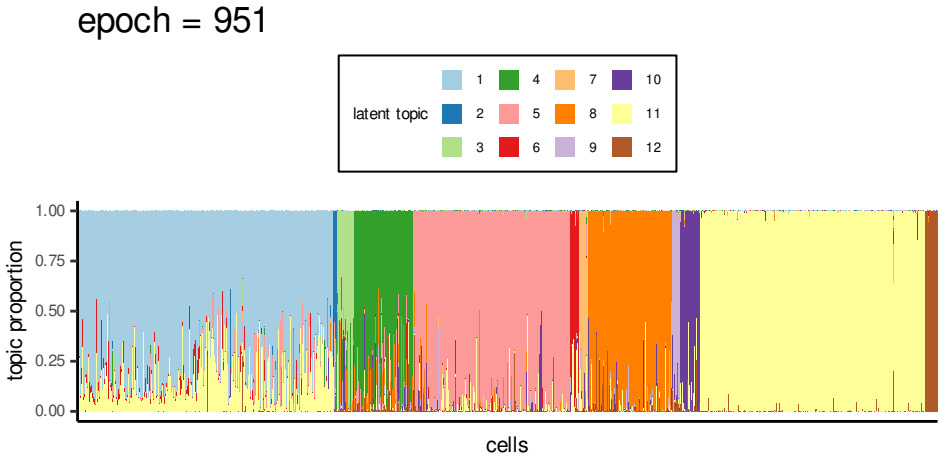
Hidden representations (mixing proportions)



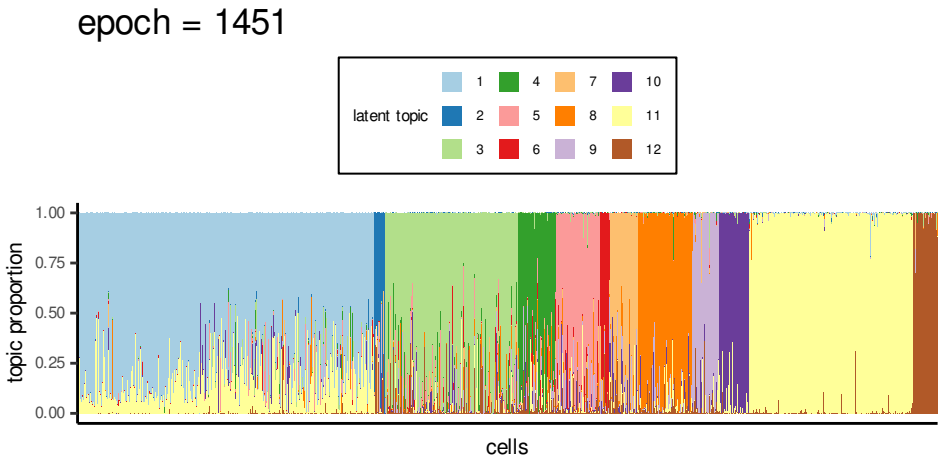
Hidden representations (mixing proportions)



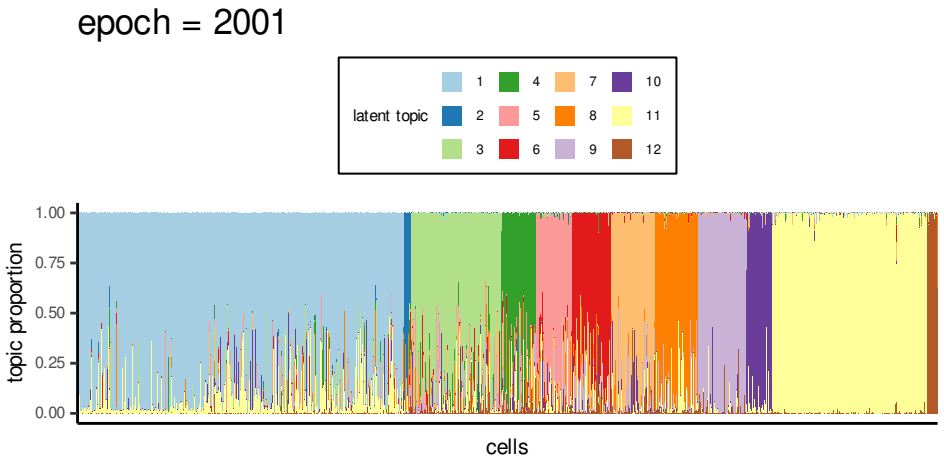
Hidden representations (mixing proportions)



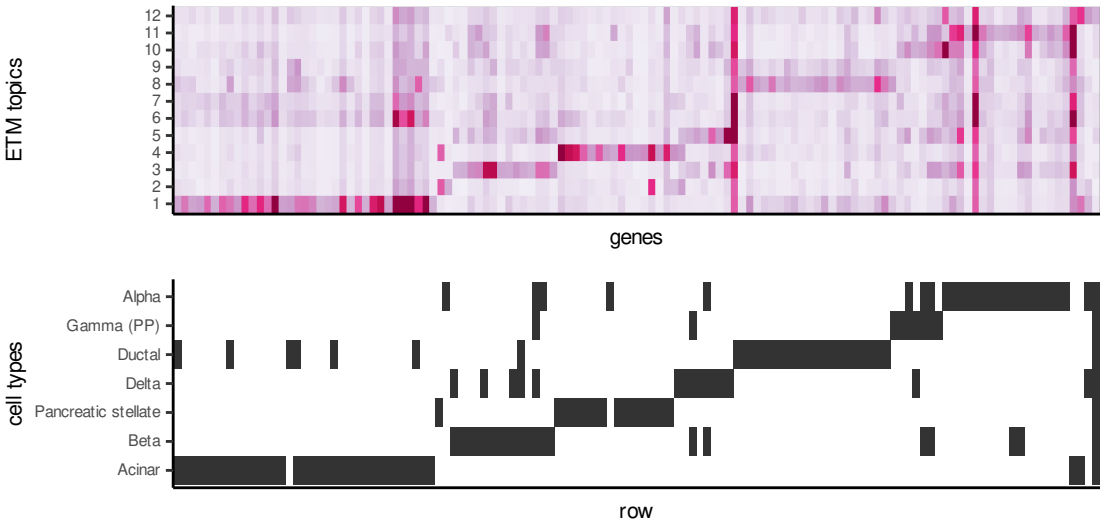
Hidden representations (mixing proportions)



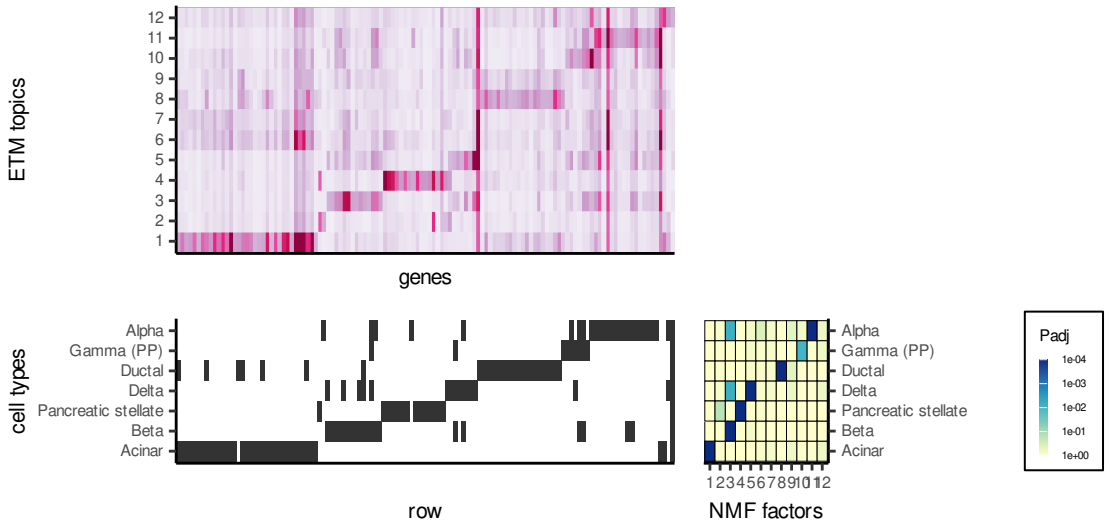
Hidden representations (mixing proportions)



Annotate factors to cell types by enrichment (`fgsea`)



Annotate factors to cell types by enrichment (`fgsea`)



Discussions on latent topic modelling

- Most cells predominantly belong to one topic (one colour). Why?
- If we model cells as a mixture of cell topics, we can capture doublets or triplets.
- The underlying generative model assumes no sequencing depth! This can help avoid batch-specific differences in practice.
- VAE offers a flexible framework with which our scientific hypothesis can be formulated in a probabilistic language (`torch`).
- Potentially, this purely-unsupervised learning framework can be combined with supervised, semi-supervised learning models.